

Chapter 1 Introduction

This chapter introduces ShellSentry, a secure web-based system that translates natural-language requests into validated Bash commands and executes them on remote Linux hosts. Section 1.1 presents the problem of safe, large-scale shell automation and the gaps in existing approaches. Section 1.2 outlines the project's objectives, including its scope, target audience, impact, limitations, and assumptions. Section 1.3 lists the contributions of this work, and Section 1.4 sketches the structure of the remainder of the report.

1.1 Problem Statement

Modern organizations operate dozens to hundreds of Linux servers that must be queried, monitored, and configured through Bash commands. Tasks such as listing active connections, checking disk space, sampling running services, or backing up a database typically require an administrator to write the right command, target the right host, and authenticate over SSH — and then repeat that work across the fleet. The skill barrier excludes most analysts, auditors, and junior staff from operations they would otherwise be responsible for, while introducing real risk: a single mistyped flag on a production server can cause downtime, data loss, or open a security gap.

Large Language Models (LLMs) offer an attractive shortcut. Tools such as Shell-GPT, GitHub Copilot in the CLI, and Open Interpreter can convert plain English into Bash, lowering the barrier dramatically. However, allowing LLM output to run on remote hosts introduces a new category of risk. Hallucinated destructive commands (`rm -rf /`, `dd if=...`), prompt-injection attacks that override the system prompt, privilege abuse through `sudo`, fork bombs, and pipe-to-shell exfiltration techniques are all documented failure modes of LLM-driven automation [4].

There is, today, no widely adopted open framework that combines a natural-language interface, multi-layer validation of LLM output, host-aware prompting, retrieval grounding against a trusted knowledge base, and auditable parallel execution over SSH. This is the gap that ShellSentry addresses. Rather than trusting the LLM, ShellSentry treats every model output as untrusted by default and gates it behind input rules, a curated retrieval layer, a whitelist/blacklist command validator, and a read-only execution policy.

1.2 Project Objectives

This project aims to design and implement a secure, web-based system that converts natural-language requests into validated Bash commands and executes them safely on one or more remote Linux hosts.

1.2.1 Scope

The project delivers a functional prototype comprising a Flask-based web interface, an authentication layer, an input-validation layer, a host-context probing mechanism over SSH, a Retrieval-Augmented Generation (RAG) layer built on a curated Bash knowledge base, and an OpenAI-compatible LLM client. It also includes a multi-rule command validator operating in a strict read-only mode, a parallel SSH executor based on Paramiko, an audit logging system, and a result-formatting layer featuring a secondary LLM pass to generate plain-language explanations of execution outcomes. Evaluation is performed using a controlled set of natural-language prompts executed against virtual Linux hosts.

1.2.2 Target Audience

The system is intended for system administrators, security analysts, junior operations staff, and IT auditors who require fast, plain-language access to information across multiple Linux hosts without manually writing Bash commands. It is also well suited for academic and educational environments — particularly courses focused on Linux administration, applied AI, and cybersecurity — as the tool presents the complete pipeline from natural-language input to validated command generation within a single response.

1.2.3 Impact

ShellSentry was developed within the context of the University of Bahrain, but its design is broadly applicable. Its effectiveness increases alongside the growth of the host fleet, as parallel SSH execution, host-aware prompting, and the default read-only mode scale linearly with the number of targets. The project also serves as an educational reference for implementing multi-layer defenses around an LLM in a security-sensitive environment—an architectural approach that is becoming increasingly relevant as more enterprises adopt agentic LLM workflows.

1.2.4 Limitations

The current prototype is designed primarily for read-only inspection and lightweight scripting tasks. State-changing commands are blocked by default; enabling write operations requires lowering `READ_ONLY_EXECUTION` and is not recommended outside controlled environments. The system supports Linux hosts exclusively through SSH, while Windows targets via WinRM remain outside the project scope. Natural-language input is mainly supported in English, without any other language support for any of the available operations. The accuracy of generated commands also depends on both the selected LLM and the breadth of the curated knowledge base, as smaller models may occasionally generate commands that are rejected by the validator.

1.2.5 Assumptions

The design assumes that target hosts run a modern Linux distribution with `bash`, `ss`, `systemctl`, and `coreutils` available. It also assumes that SSH access is configured using SSH keys or per-server credentials stored in a `.env` file located outside the source tree, and that the Flask service operates within a controlled, authenticated environment. The LLM endpoint is expected to use an OpenAI-compatible API, such as OpenAI, Groq, or a local Ollama gateway. Additionally, the RAG knowledge base is assumed to contain only trusted, manually authored command examples.

1.3 Contribution

This project makes the following five contributions:

- 1. A defense-in-depth pipeline for LLM-generated shell commands.**
Five independent layers—user-input rules, a constrained system prompt, retrieval grounding, a command whitelist/blacklist with per-command handlers, and a strict read-only execution policy—operate together to ensure that any bypass at one level is mitigated by the next.
- 2. Host-aware prompting through lightweight SSH probing.**
Before each generation request, ShellSentry gathers per-host snapshots (`uname -a`, active systemd units, and listening TCP/UDP sockets) in parallel and injects them into the LLM prompt, enabling the model to select tools and paths that accurately reflect the services running on each target.
- 3. Retrieval-Augmented Grounding for Bash automation.**
A curated knowledge base of trusted description-to-command pairs is encoded using `sentence-transformers/all-MiniLM-L6-v2` and indexed in FAISS with inner-product (cosine) similarity. The top-*k* examples are appended to every LLM prompt, reducing hallucinations and improving the auditability of generated commands.
- 4. An audit-first execution architecture.**
Every request, generated command, target list, and per-host result is stored as an `ExecutionLog` entry in the database and recorded in a file log. Multi-line scripts are automatically saved on the remote host under `$HOME/ShellSentryScripts/`, allowing later listing, re-execution, and AI-generated explanations through the dashboard.
- 5. A reproducible evaluation across security, performance, and model dimensions.**
The system was evaluated using eight security test categories, four LLM model configurations, two execution modes (read-only and open), and an industry baseline comparison against Shell-GPT, all using the same set of natural-language prompts.

1.4 Report Outline

The remainder of this report is organized as follows. Chapter 2 reviews the literature and prior systems related to natural-language shell automation, prompt injection, and Retrieval-Augmented Generation. Chapter 3 describes the project management approach, including the selected Software Development Life Cycle, the risk register, and the activity plan. Chapter 4 presents the system requirements, covering requirements elicitation, functional and non-functional requirements, and system-level models such as the architecture and ERD. Chapter 5 details the system design, including the database schema, user-interface design, and pseudocode for the core procedures. Chapter 6 discusses the implementation and evaluation results, including security testing, performance measurements, model-comparison experiments, and a comparison with a real-world solution. Finally, Chapter 7 concludes the report with a summary of findings, project limitations, and recommendations for future work.

Chapter 2 Literature Review

This chapter positions ShellSentry within the existing literature on Large Language Models (LLMs), Bash command generation, and IT operations automation. Section 2.1 presents the background of Natural Language to Bash (NL2SH) systems and the limitations they face. Section 2.2 reviews three recent academic studies that are directly relevant to ShellSentry's design. Section 2.3 summarizes the main findings, and Section 2.4 explains how those findings shape our project.

2.1 Background

Natural Language to Bash (NL2SH) systems allow users to describe Linux tasks in plain language and automatically receive runnable Bash commands. Traditional Linux administration requires memorizing command syntax and understanding dozens of utilities, which raises the barrier for beginners and slows experienced administrators when the same operation must be performed across many hosts. The recent generation of Large Language Models (LLMs) has materially improved NL2SH because instruction-tuned models can understand user intent and generate syntactically correct shell commands for a wide range of tasks.

Despite these advances, AI-generated Bash output remains unreliable in three ways. First, models still produce syntax errors and select the wrong utility for the task. Second, they hallucinate destructive operations: a benign-sounding request can return a command that deletes data or modifies system state in unintended ways. Third, even when the output is technically valid, there is no guarantee that it is functionally correct — two different commands can produce the same result, and a command that looks right may behave differently from what the user actually wanted.

Recent research has therefore shifted from pure generation toward validation, refinement, and controlled execution. The studies reviewed in Section 2.2 each address one of these dimensions, and ShellSentry builds on all three by combining LLM-based Bash generation with multi-layer validation, secure SSH-based execution, audit logging, and managed scheduling.

2.2 Related Work

This section reviews three recent studies that informed the design of ShellSentry: ScriptSmith [1], an LLM framework for automated Bash script generation and refinement in IT operations; an empirical study on LLM-supported Natural Language to Bash translation [2]; and an LLM-as-a-Judge framework for automatic code validation and refinement [3]. Together they cover the three pillars of secure NL2SH — generation, evaluation, and refinement — and the gap each leaves behind motivates the design choices in our project.

2.2.1 ScriptSmith — Automated Bash Script Generation, Assessment, and Refinement [1]

ScriptSmith is an LLM-based framework that targets IT operations through automated Bash script generation and refinement. The system generates Bash scripts for incident-remediation tasks, evaluates their correctness, refines failing scripts automatically, and stores verified scripts in a reusable catalogue for later reuse. The authors report measurable improvements in Bash generation accuracy and reduced manual scripting effort in Site Reliability Engineering (SRE) environments.

The strengths of ScriptSmith are its operational focus and its closed feedback loop: generation, automated assessment, and refinement work together rather than in isolation, and the reusable catalogue reduces repeated work for recurring incidents. The framework demonstrates that LLM output for shell automation is viable when paired with an evaluation step.

The limits of ScriptSmith are equally clear. The framework concentrates on generation and evaluation and does not address secure execution: there is no authentication layer, no SSH-based multi-host orchestration, no audit log of what ran where, and no scheduling mechanism. The system also depends on human expert review before scripts are used in production, which limits its operational autonomy.

ShellSentry adopts ScriptSmith's principles of generation plus validation, but extends them with a defense-in-depth pipeline, authenticated multi-host SSH execution, an ExecutionLog audit trail, and a Safe Cron Mode for managed scheduling.

2.2.2 LLM-Supported Natural Language to Bash Translation [2]

This study focuses on the translation problem itself: how to measure whether an LLM-generated Bash command actually does what the user asked. The authors construct a verified dataset of instruction-command pairs and propose a functional-equivalence heuristic that combines

command execution with LLM evaluation of the resulting outputs, achieving high accuracy at deciding whether two Bash commands perform the same task.

The strongest contribution of this paper is its evaluation methodology. The work argues — and demonstrates empirically — that Bash correctness must be measured functionally rather than syntactically, because multiple commands can achieve the same result through different utilities. The paper also documents the most common LLM failure modes in this domain: incorrect formatting, wrong utility selection, and invalid Bash syntax.

The paper also makes an important security observation that directly motivates ShellSentry's design. During testing, the authors observed hallucinated commands that damaged Docker environments — concrete evidence that LLM-generated Bash must not be executed without validation. Even on advanced models, accuracy stayed below the level required for unattended deployment.

The study's limit is that it remains a benchmarking and translation study rather than an operational platform. It does not address authentication, remote server management, or secure deployment.

This research strongly supports ShellSentry's security-first design, in which generated commands are validated and restricted by a whitelist, blacklist, and read-only policy before they ever reach a remote host.

2.2.3 LLM-as-a-Judge for Automatic Code Validation and Refinement [3]

This paper proposes an LLM-as-a-Judge framework for validating and refining Bash commands generated from natural language. Rather than relying on string similarity, the framework checks whether the generated code logically satisfies the requested task using bidirectional functionality matching and logic representation techniques. A reflection-agent loop then automatically rewrites incorrect commands, and the authors report significant improvements in command-refinement accuracy.

The strength of this work is its semantic validation approach. Logical equivalence is a more reliable correctness signal than text similarity, and the reflection mechanism turns a one-shot generation step into a self-correcting loop. The paper makes a strong case that NL2SH systems should treat validation as a first-class component, not an afterthought.

The principal limit is that the entire framework depends on LLM reasoning. If the judge model itself hallucinates or misinterprets the task, the verdict and the refinement can both be wrong. The study also stops at validation and refinement; secure deployment, authentication, SSH orchestration, and audit logging are out of scope.

This work supports the validation philosophy used in ShellSentry, in which every generated command is checked against a deterministic policy before execution, and only commands that pass every layer reach the SSH executor.

2.3 Summary

The reviewed studies converge on five conclusions:

1. LLMs can generate Bash effectively, but raw generation alone is not safe enough for unattended production use [1], [2], [3].
2. Functional correctness, not syntactic similarity, is the right standard for evaluating NL2SH output [2], [3].
3. Validation and refinement loops measurably improve generation quality [1], [3].
4. Hallucinated, destructive commands have been observed in real testing environments — empirical evidence that NL2SH must be paired with deterministic safety checks [2].
5. Secure operational deployment — authentication, multi-host orchestration, audit logging, and scheduling — is largely absent from the existing literature, which focuses on generation and evaluation.

2.4 Relevance to Our Project

ShellSentry inherits and extends ideas from each of the studies reviewed above. From ScriptSmith [1] it borrows the closed loop of generate → validate → refine and the principle of a reusable script catalogue, and extends them with secure remote execution, command-policy enforcement, audit logging, and managed scheduling — the operational layer that ScriptSmith leaves to its users. From the LLM-Supported Natural Language to Bash Translation study [2] it takes the evidence that LLM output must be validated before execution, since hallucinated destructive commands have already been observed in practice; this finding directly justifies ShellSentry's whitelist, blacklist, per-command handlers, and read-only execution policy. From the LLM-as-a-Judge framework [3] it takes the principle that validation should be a first-class component of the pipeline, and operationalizes that principle as a deterministic, layered validator rather than a single LLM judge — eliminating the risk that a hallucinating judge silently approves an unsafe command.

Where prior work treats NL2SH as a generation-and-evaluation problem, ShellSentry treats it as a secure-execution problem. The contribution that distinguishes this project from the literature is the integrated, defense-in-depth platform that unifies natural-language input, host-aware prompting, retrieval-augmented grounding, multi-layer command validation, authenticated multi-host SSH execution, audit logging, script archiving with re-execution, and Safe Cron scheduling — so that commands are not only generated accurately, but also executed securely, transparently, and reproducibly across diverse operational environments.

Chapter 3 Project Management

This chapter describes how the ShellSentry project was planned and carried out. Section 3.1 presents the software process model and explains how Agile development was applied to this

prototype. Section 3.2 identifies project risks and mitigation strategies. Section 3.3 presents the activity plan and scheduling artifacts used to deliver the system within the senior-project timeframe.

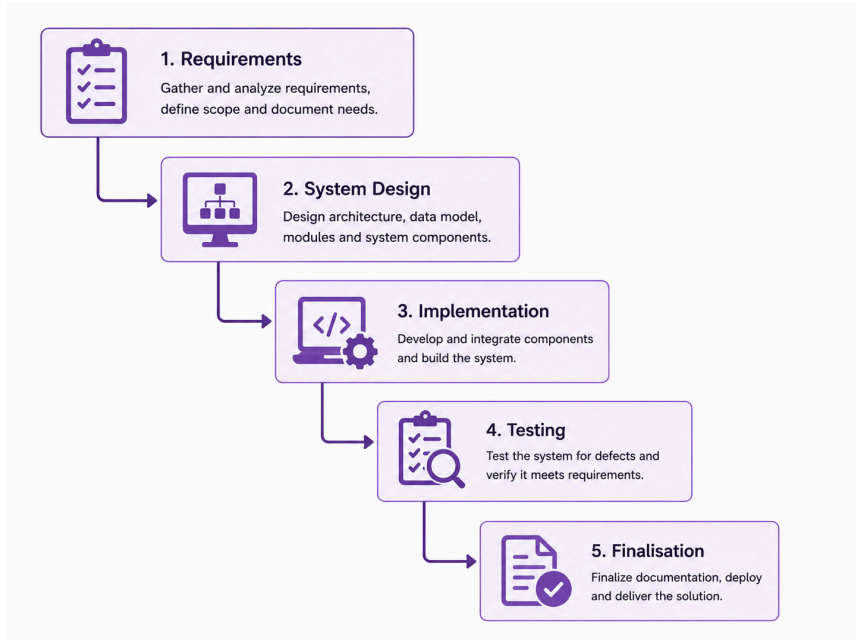
3.1 Process model

The Waterfall software development life cycle was adopted as the primary process model. In Waterfall, development progresses through sequential phases, where each phase produces approved artefacts before the next phase begins (requirements, design, implementation, testing, and deployment/documentation). This structure fits an academic senior project for several reasons.

First, the department and supervisor require clear checkpoints and stable requirements documentation (especially for Chapters 4 and 5), which Waterfall supports through formal phase outputs. Second, ShellSentry combines several sensitive concerns—remote execution, large language model (LLM) generation, and security policy enforcement—that benefit from completing threat and requirement thinking before committing to detailed design and coding. Third, the fixed calendar between September 2025 and May 2026 maps naturally onto discrete phases with predictable milestones (requirements sign-off, design freeze, feature-complete build, test campaign, final submission).

The model was applied pragmatically: minor clarifications to requirements were allowed during implementation when they did not invalidate the architecture (for example, tightening validation messages or UI wording). However, core functional boundaries (authenticated execution, SSH-based remote run, validation layers, audit logging, script archive, and safe cron workflow) were treated as baseline scope agreed before intensive coding, consistent with a Waterfall mindset.

Figure X (you insert): Phases of the Waterfall model used for ShellSentry (Requirements → System Design → Implementation → Testing → Finalisation).



3.2 Risk management

In Section 3.2, the project discusses risk management practices implemented to identify, evaluate, and reduce potential risks that could affect the system’s performance, security, and reliability. Risk management is essential to ensure stable workflow execution, protect sensitive information, and prevent operational failures. The analysis focuses on technical and security-related risks such as API instability, SSH connectivity problems, destructive command execution, prompt injection attacks, cron schedule abuse, and credential exposure. Appropriate mitigation strategies are proposed to minimize these risks and maintain safe system operations.

Table X presents the project’s risk management analysis by outlining the identified risks, their severity levels, possible impacts, and corresponding mitigation strategies. The table demonstrates how preventive measures such as command validation, secure credential handling, audit logging, retry mechanisms, connectivity checks, and strict execution controls help reduce vulnerabilities and improve overall system stability and security.

Table X. Identified risks and mitigation plans for ShellSentry

Risk	Level	Impact	Mitigation

LLM API instability (timeouts, rate limits, provider outages)	Medium	Generation failures block the main workflow	Client-side retries/backoff; graceful error responses in the UI; fall back to regex-based routing where applicable; keep logs for diagnosis
SSH connectivity and credential issues (keys, passwords, firewalls)	Medium	Partial or failed multi-host runs	Pre-flight reachability checks; parallel execution so one host does not block others; documented <code>.env</code> setup
LLM generates state-changing or destructive commands (e.g. <code>rm -rf</code> , <code>mkfs</code> , <code>dd if=...</code>)	High	Data loss or host compromise	Command validator (whitelist/blacklist/restricted patterns); read-only mode default; normalization before execution; audit logging
Prompt injection — malicious or misleading natural-language input crafted to bypass intent or policy	High	Unsafe execution or policy bypass	<code>SecurityLayer</code> input checks; intent routing + regex blocks (e.g. destructive cron phrases); validation after LLM output; no blind trust of model text
Cron schedule abuse — scheduling archived scripts too aggressively, wrong expressions, or attempting bulk removal of crontab	Medium–High	Unwanted automation or denial patterns	Safe Cron mode; list/schedule only for managed entries; destructive removal phrases blocked; strict cron expression checks; script must exist and pass re-validation

SSH credential theft or exposure (passwords/keys in logs, screenshots, or misconfigured <code>.env</code>)	High	Unauthorized access to lab servers	Secrets only in environment/config (not in repo); avoid logging secrets; document secure handling; least-privilege SSH user for demos
--	------	------------------------------------	---

3.3 Project activities plan

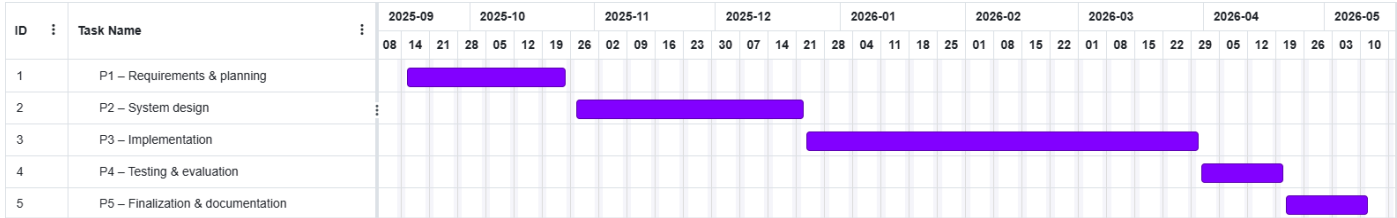
The activity plan covers 34 weeks, from Monday 15 September 2025 to Monday 11 May 2026, aligned with the senior project’s official teaching period and submission window. Work was divided into five Waterfall-style workstreams so that each stage produced clear outputs before the next: (1) requirements and planning—clarifying scope, security boundaries, and lab constraints; (2) system design—architecture, data model, and execution flow; (3) implementation—building and integrating the Flask application, LLM and SSH components, and the script archive and safe cron features; (4) testing and evaluation—functional runs on multiple hosts, security-oriented checks, and performance observations for the report; and (5) finalisation—completing Chapters 5–7, figures and references, the poster, and submission materials (including any department requirements for appendices or deliverables). The following table and Gantt chart summarise how these workstreams were scheduled across the project timeline.

Table X. ShellSentry project activity plan (summary by phase)

Phase	Period (inclusive)	Main outputs
P1 – Requirements & planning	15 Sep 2025 – 24 Oct 2025	Problem refinement; initial objectives and scope; environment setup; draft requirement list; preliminary architecture sketch
P2 – System design	27 Oct 2025 – 22 Dec 2025	Architecture diagram; module responsibilities; database design (<code>User</code> , <code>ExecutionLog</code>); UI wireframes; pseudocode for execution pipeline
P3 – Implementation	23 Dec 2025 – 30 Mar 2026	Working Flask application; authentication; <code>/api/execute</code> orchestration; SSH executor; LLM client + RAG integration; archive and safe cron paths

P4 – Testing & evaluation	31 Mar 2026 – 20 Apr 2026	Functional tests on multi-host scenarios; security-focused tests; performance timings on core procedures; results tables for Chapter 6
P5 – Finalisation & documentation	21 Apr 2026 – 11 May 2026	Final report chapters and revisions; figures and references polish; poster/demo preparation; submission checklist (including appendices/CD requirements per department guidance)

Gantt chart



3.4 Chapter Summary

This chapter presented the project management approach used for the development of ShellSentry, covering the planning, risk management, and scheduling activities that guided the project from initial concept to prototype completion. The Waterfall software development model was selected as the primary process framework due to its structured, phase-based nature and its suitability for academic projects requiring clearly defined deliverables and milestone checkpoints. The chapter also identified the major technical and security risks associated with the project, including LLM instability, unsafe AI-generated commands, SSH connectivity failures, prompt injection, cron misuse, and credential exposure, together with mitigation strategies designed to reduce their impact. Finally, the project activity plan outlined the development timeline across requirements analysis, system design, implementation, testing, and final documentation, providing a structured roadmap for delivering the ShellSentry prototype within the senior-project timeframe.

Chapter 4 Requirement Collection and Analysis

This chapter defines what ShellSentry must do and how those needs were established. It begins with requirement elicitation (Section 4.1), summarising how stakeholder input and technical constraints shaped the specification. Section 4.2 formalises functional and non-functional

requirements derived from that process. Section 4.3 presents system models—a high-level architecture of the web application, validation layers, LLM and SSH services, and the relational schema that supports authentication and audit logging. Together, these artefacts provide a traceable bridge between the problem statement in Chapter 1 and the design and implementation chapters that follow.

4.1 Requirement Elicitation

Requirements related to security, traceability, and safe automation were gathered through semi-structured interviews with two participants. Participant 1 provided an extended written response from a cybersecurity perspective. Participant 2 answered the same themes in the interview using concise numbered points rather than long narrative prose. Together, the interviews informed ShellSentry’s emphasis on defence in depth, read-only defaults, deterministic validation, and auditability. Table 4.1 summarises the interview questions and both participants’ answers.

Table 4.1. Requirement elicitation — questions and participant responses

#	Question	Participant 1 (Umer Khan)	Participant 2 (anonymous)
1	What are the main risks when manually executing Bash commands across multiple remote servers?	Risks include executing commands against the wrong host or environment, privilege misuse, lack of validation/safeguards, inconsistent execution across the fleet, and insufficient logging and auditing. Even skilled administrators can cause outages or security exposure with a single mistyped command.	The biggest risks are running high-privilege commands without proper verification, and lack of clear traceability for what changed on each system.
2	How does human error contribute to incidents in system administration?	Human error remains a major cause of operational and security incidents—e.g. accidental deletion or modification of critical files, incorrect permission changes, running commands with elevated privileges, exposing secrets in scripts or	Human error is a leading cause of incidents: harmful commands, permission misconfigurations, and running commands in the wrong environment (e.g. production instead of testing).

		logs, and misconfigurations that weaken security. Often the issue is scale, complexity, and time pressure, not only lack of skill.	
3	What new risks appear when an LLM is used to generate shell commands?	New risks include prompt injection, unsafe command generation, hallucinated syntax or logic, hidden chaining of dangerous operations, privilege escalation opportunities, and indirect trust in untrusted input. The output is probabilistic, which changes the security model compared with traditional automation.	Introducing an LLM adds risks such as prompt injection, and commands that look valid but can still cause serious system impact.
4	In practice, which threats are most realistic for LLM-assisted shell execution?	Privilege abuse and unsafe command execution are among the most immediate risks—a technically valid command can still be harmful if run with excessive permissions. Prompt injection is highly relevant when the model processes external or untrusted content; command injection matters if user input is concatenated into shell commands without strict sanitisation and parsing.	Privilege abuse and command injection are among the most realistic threats—a small mistake with elevated privileges can quickly become a serious incident.
5	Is whitelist/blacklist validation enough on its own to secure generated commands?	Not sufficient. Blacklists are especially weak because bypasses continually appear. A safer design combines strict allowlists, parameterised templates, RBAC/least privilege, sandboxing,	Whitelist/blacklist checks are useful but not sufficient alone; they should be combined with sandboxing and least-privilege access.

		human approval, simulation/dry-run, strong auditing, and defence in depth.	
6	How important is read-only-by-default behaviour for AI-assisted administration?	Extremely important. Read-only default dramatically reduces blast radius and is one of the best architectural decisions for such systems. Destructive or state-changing operations should require elevated approval, extra validation, and explicit operator confirmation.	Highly important—it significantly reduces risk and keeps the system focused on visibility and analysis rather than unchecked modification.
7	What logging and auditing should stakeholders expect before trusting this class of system (e.g. in an enterprise or SOC-oriented context)?	Expect immutable audit logs, user attribution, timestamps, original prompts, generated commands, approval history, execution results, target-host tracking, rollback/recovery visibility, and alerting for high-risk operations. Traceability and accountability are critical.	For a SOC-style context, expect logging of operator identity, IP addresses, timestamps, and execution outcomes (success/failure). <i>(Participant 1 gave a broader enterprise list above.)</i>
8	What minimum security controls would you recommend for a responsible educational proof of concept?	Use sandboxed execution only, no production access, least-privilege execution, read-only default, strict command allowlists, human approval before execution, complete logging, and clear documentation of limitations. Demonstrate blocked malicious prompts in evaluation to show security thinking, not only happy-path behaviour.	Minimum controls should include: execution in an isolated sandbox or container, strictly limited privileges, blocking dangerous commands by default, full execution logging, and an approval step for sensitive operations.

Both participants stressed that convenience must not replace controls: validation layers, least privilege, read-only defaults, and traceable execution are central. These themes map directly to ShellSentry's functional and non-functional requirements in Section 4.2 (e.g. authenticated execution, deterministic policy enforcement after LLM generation, read-only configuration, and persistent execution logs).

4.2 System Requirements

This section specifies the functional requirements and the non-functional requirements. Functional items describe what the system does; non-functional items describe how well it must do it under constraints relevant to security-sensitive automation.

4.2.1 Functional Requirements

Natural-language to Bash generation

The system shall convert user natural-language requests into executable Bash commands or multi-line Bash scripts using a Large Language Model (LLM). The generated output shall contain executable shell commands only and shall be prepared for execution on remote Linux hosts.

Host-aware context generation

The system shall probe selected Linux hosts over SSH to collect contextual information such as operating-system details, running services, and listening network sockets. This contextual information shall be supplied to the LLM together with the user request in order to improve environment-aware command generation.

Retrieval-augmented grounding

The system shall retrieve the top-k semantically relevant command examples from a curated Bash knowledge base using semantic similarity search and shall inject them into the LLM prompt to reduce hallucinations and improve consistency with trusted command patterns.

Deterministic safety validation before execution

The system shall validate every generated command and every archived script prior to execution using deterministic policy-enforcement mechanisms including whitelists, blacklists, restricted-pattern detection, and read-only execution enabled by default with configurable override support. Commands that fail validation shall be blocked from execution.

SSH execution

The system shall remotely execute approved Bash commands and scripts on selected Linux hosts through SSH connections. The execution subsystem shall support pre-flight reachability

checks and isolated per-host execution results so that failure on one host does not prevent execution on others.

SSH authentication management

The system shall support multiple SSH authentication mechanisms including password-based authentication, SSH private-key authentication, SSH-agent usage, and per-server credential overrides defined through system configuration.

Multi-server orchestration

The system shall support concurrent execution across multiple configurable Linux hosts defined through environment-based configuration. The system shall execute commands in parallel to reduce delays caused by sequential execution.

Script archival and re-execution

The system shall automatically archive generated multi-line Bash scripts on remote hosts for future reuse. The system shall support listing archived scripts, validating them before execution, re-running saved scripts, and retrieving script content for explanation purposes.

Safe Cron scheduling

The system shall support controlled cron scheduling for archived scripts by allowing users to list managed cron entries and schedule existing archived scripts using validated cron expressions. Destructive cron operations such as deleting or clearing crontab entries shall be blocked.

Audit logging and reporting

The system shall maintain persistent execution records including the original natural-language request, generated command, selected target servers, execution status, timestamps, and execution results. The system shall additionally generate structured technical reports and human-readable summaries for executed operations.

Configurable execution and security policies

The system shall support configurable execution policies through environment-based configuration, including read-only execution enabled by default with configurable override support, privileged/root execution disabled by default with configurable administrative override, configurable archive behaviour, configurable cron-management settings, and configurable LLM provider settings.

4.2.2 Non-functional requirements

Security

Defence in depth: input sanitisation (SecurityLayer), LLM-assisted and regex-based intent routing with hard blocks for destructive cron phrasing, policy validation after generation, read-only execution enabled by default with configurable override support, and SSH execution under configured credentials. Secrets shall reside in environment configuration, not source code.

Confidentiality and least privilege

SSH credentials and API keys shall not be embedded in the repository; remote execution shall use the minimum OS privileges assumed in deployment documentation.

Reliability and resilience

The LLM client shall implement retries/backoff for transient API failures (e.g. timeouts, rate limits). SSH execution shall tolerate partial host failures and surface per-host errors clearly in API responses.

Performance and scalability (within prototype bounds)

Host probing and execution shall run in parallel across targets to avoid sequential bottlenecks. Acceptable latency is bounded by LLM and network behaviour; the architecture shall not impose unnecessary serial stages beyond policy checks.

Usability

The web dashboard shall present a natural-language summary, optional AI explanations, and an expandable technical report so both technical and less-specialised users can interpret outcomes.

Maintainability and auditability

The codebase shall separate routing, validation, SSH, LLM, RAG, and formatting concerns; every execution path shall be traceable via logs and database records for post-incident review.

4.3 System Models

This section presents the core system models used in ShellSentry, including the overall architectural model and the database entity relationship model. These models illustrate how the system components interact to support secure AI-assisted remote administration and how persistent data is managed for authentication and auditability.

4.3.1 System architecture

Figure 4.1 depicts ShellSentry's major components and control flow. End-to-end flow: the authenticated user submits natural language and an optional server list via the browser to Flask (POST /api/execute). The Security Layer validates raw text. Execution routing (resolve_execute_route) classifies intent using an LLM classifier when configured, with regex fallback and safety blocks for forbidden cron behaviour. Depending on the route, the system may call SSHExecutor for managed cron or script-archive operations, or follow the standard pipeline: probe_host_context → RAG retrieval → LLM generate_command → CommandValidator → normalize_for_execution → execute_on_servers. Results pass through the result formatter; optional summarize_execution_report / explain_script calls improve readability. SQLite stores users and ExecutionLog entries.

[Insert Figure 4.1 — System architecture]

Caption suggestion: Figure 4.1. ShellSentry system architecture (web client, Flask API, security and routing layers, RAG and LLM services, command validator, parallel SSH executor, audit database).

4.3.2 Entity relationship diagram (ERD)

ShellSentry uses a relational database for authentication and audit trails. Figure 4.2 shows the logical schema.

User stores credentials and profile fields (username, email, password hash, is_admin, created_at).

ExecutionLog stores each run's original_request, generated_command, serialised target_servers and execution_results, execution_status, timestamp, and denormalised username, linked by user_id as a foreign key to User.

[Insert Figure 4.2 — ERD]

Caption suggestion: Figure 4.2. Entity relationship diagram for ShellSentry (User and ExecutionLog).

4.4 Chapter Summary

This chapter presented the requirement analysis for ShellSentry, including stakeholder-driven requirement elicitation, formal functional and non-functional requirements, and the core system models. The elicitation process emphasized security, validation, auditability, and controlled automation, which directly shaped the system requirements. The chapter also introduced the

high-level system architecture and database model, providing the foundation for the detailed design in Chapter 5 and the implementation and evaluation presented in Chapter 6.

Chapter 5 System Design

This chapter describes the logical design of ShellSentry and explains how the user requirements presented in Chapter 4 are realised through data structures, user interaction mechanisms, and backend processing, prior to the implementation and testing details discussed in Chapter 6. The design follows a defence-in-depth approach in which natural-language requests are never executed blindly. Instead, each request passes through multiple security and validation layers, including authentication, input validation, intent classification, retrieval-augmented prompting, deterministic command-policy enforcement, and parallel SSH execution, with all outcomes recorded for auditing purposes.

The chapter is organised as follows. Section 5.1 outlines the scope of the design and introduces the main software modules. Section 5.2 presents the database design, including the purpose of local persistence and the schemas for the `User` and `ExecutionLog` entities. Section 5.3 discusses the user interface and interaction design. Section 5.4 presents the core procedures and pseudocode for the execution pipeline, including the Safe Cron workflow, script-archive routes, and the standard natural-language-to-Bash execution path. Section 5.5 maps the system requirements to the corresponding design artefacts, and Section 5.6 concludes the chapter.

5.1 Introduction

This chapter presents the logical design of ShellSentry prior to the implementation and evaluation discussion in Chapter 6. The design translates the functional and non-functional requirements identified in Chapter 4 into three complementary views: persistent data managed by the web application (Section 5.2), user-facing interfaces and navigation (Section 5.3), and core procedures and algorithms governing natural-language routing, validation, remote execution, and reporting (Section 5.4).

ShellSentry's architecture is deliberately modular. The Flask application separates HTTP orchestration (routes), intent resolution (`execution_router`), input hygiene (`SecurityLayer`), LLM interaction (`LLMClient`), retrieval grounding (`RagPipeline`), deterministic command policy (`CommandValidator`), parallel SSH operations (`SSHExecutor`), response shaping (`result_formatter`), and persistence (models). Remote Linux hosts remain the system of record for command effects and for archived scripts under the configurable directory (for example `$HOME/ShellSentryScripts`); the local database primarily supports authentication and auditability, consistent with the interview findings on traceability and accountability.

Section 5.2 documents the relational schema used by the prototype. Sections 5.3 and 5.4 (developed in the remainder of this chapter) detail the interface design and algorithmic

behaviour that realise defence in depth, read-only defaults, and auditable multi-host execution described in Chapters 1 and 4.

5.2 Database Design

This section presents the database design used in ShellSentry to support user authentication and execution audit logging. The prototype uses a lightweight relational database implemented through Flask-SQLAlchemy, providing persistent storage for user account management and execution history. The database was intentionally designed with a minimal schema focused on security, traceability, and maintainability, while operational data such as live server state, archived scripts, and cron configurations remain managed on the remote target systems rather than duplicated locally.

5.2.1 Purpose of the database in ShellSentry

The prototype uses a relational database accessed through Flask-SQLAlchemy with configuration supplied via `SQLALCHEMY_DATABASE_URI` (defaulting to SQLite for development). The database fulfils two roles aligned with Chapter 4’s non-functional requirements on security and auditability.

First, it stores account credentials for the web tier: user registration data and password hashes underpin Flask-Login session authentication so that only authenticated users may invoke execution APIs. Second, it stores an execution audit trail for each remote operation logged through the application: who requested an action, what natural-language prompt was supplied, what command string was approved for execution (or what built-in operation was recorded), which hosts were targeted, aggregate status, and structured per-host results where applicable.

Operational state on remote servers—live process lists, disk contents, or the full corpus of archived scripts—is not mirrored into this database by design. That boundary keeps the local schema small, reduces sensitive duplication, and matches the educational prototype scope stated in Chapter 1. Long-term script storage and cron definitions remain on target hosts under SSH-accessible paths and the user’s crontab, as implemented in the SSH executor layer.

5.2.2 Logical schema and data dictionary

The logical schema comprises two entities: User and ExecutionLog, linked by a one-to-many relationship each user may have many execution log rows. Figure 4 in Chapter 4.3.2 illustrates their relationship. Table 5.1 and Table 5.2 summarise field-level metadata.

Table 5.1 User entity — data dictionary

Attribute	Type (logical)	Constraints	Description
-----------	----------------	-------------	-------------

id	Integer	Primary key	Surrogate identifier for the account.
username	String(80)	Unique, not null	Login name displayed in logs and UI.
email	String(120)	Unique, not null	Contact identifier used at registration.
password_hash	String(255)	Not null	Stores a werkzeug password hash; plaintext passwords are never persisted.
created_at	DateTime	Default UTC now	Approximate account creation time for administration.
is_admin	Boolean	Default false	Reserved flag for administrative distinction if extended beyond the prototype.

Table 5.2. ExecutionLog entity — data dictionary

Attribute	Type (logical)	Constraints	Description
id	Integer	Primary key	Unique identifier for each logged execution event.
user_id	Integer	Foreign key → User.id, not null	Links the row to the authenticated principal.
username	String(80)	Not null	Denormalised username for readable audit exports without joining.
original_request	Text	Not null	User's natural-language request as submitted to the API.
generated_command	Text	Not null	Command string or descriptive label for built-in routes (e.g. listing cron or scripts) recorded as executed for audit.
target_servers	Text	Not null	Serialized representation of the host list (JSON string)

			in implementation).
<code>execution_status</code>	String(20)	Not null	High-level outcome category (e.g. success, failed, partial) for filtering and reporting.
<code>execution_results</code>	Text	Nullable	Serialised per-host outcomes and detail (JSON string in implementation).
<code>timestamp</code>	DateTime	Default UTC now	Wall-clock time of logging for chronological audit review.

The relationship between User and ExecutionLog is mandatory on the child side: every log row references exactly one User. Deleting a user record would normally require a policy decision (cascade delete versus retaining anonymised logs); the prototype assumes standard relational integrity as enforced by the ORM and database.

5.2.3 Design decisions and scope boundaries

SQLite (or any URL supplied through configuration) is adequate for the senior-project prototype because concurrent writers are limited to a single Flask process or modest deployment, and the volume of ExecutionLog rows remains bounded by demonstration and testing activity. Production deployments would typically migrate to a server-managed RDBMS and tighten backup and access control around audit tables.

Fields such as `target_servers` and `execution_results` are stored as text-serialised structured data to avoid premature normalisation of host lists and variable-length SSH transcripts into separate tables; this trade-off favours implementation speed and readable exports for the report and appendix material.

Finally, the database does not store LLM API keys, SSH passwords, or private keys—those remain in environment configuration outside version control, consistent with Chapter 1 assumptions and Section 4.2 security requirements.

5.3 User interface and interaction design

The ShellSentry front end is implemented as server-rendered HTML (Jinja2 templates) with vanilla JavaScript on the dashboard for asynchronous calls to POST `/api/execute`. Interaction design follows the principle that execution is never anonymous: all operational screens require an authenticated session. Figures referenced below should be filled with screenshots or wireframes from your running prototype.

5.3.1 Landing, login, and registration

The landing route (`/`, `index.html`) welcomes unauthenticated visitors and directs authenticated users to the dashboard. Login (`/login`, `login.html`) collects username and password; failed authentication surfaces a flash message without disclosing which field failed. Registration (`/register`, `register.html`) collects username, email, password, and confirmation; the server validates uniqueness and password confirmation before creating a User record with a hashed password. Successful registration redirects to login with a success message. Together, these views satisfy the authenticated access prerequisite for every execution path and support the audit logging requirement by binding future `ExecutionLog` rows to a known principal.

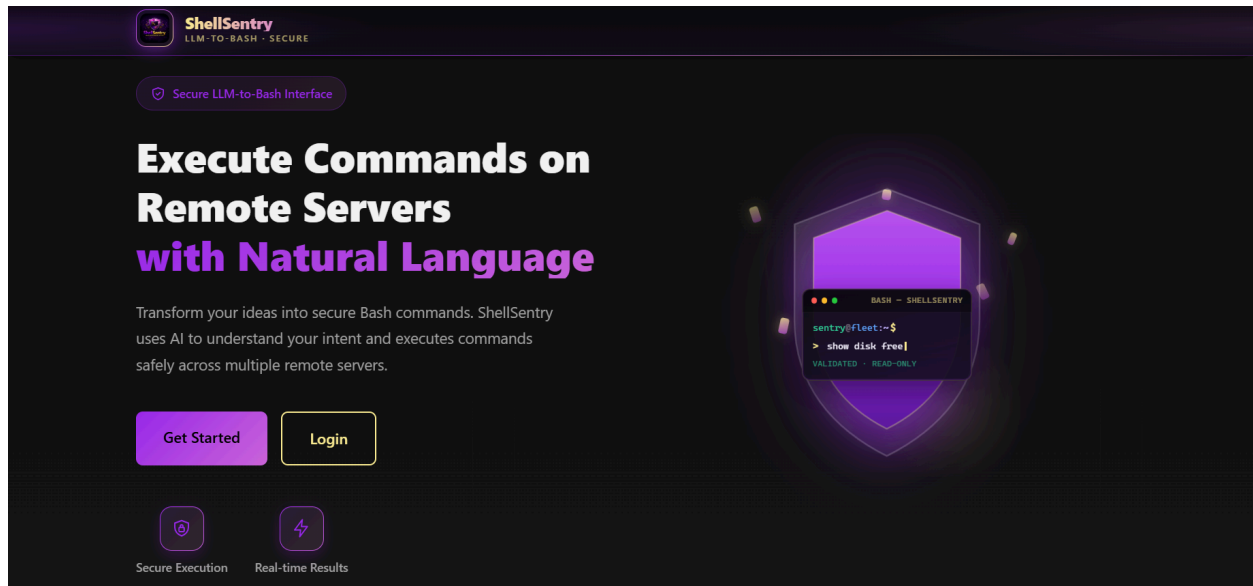


Figure : Main Page User Interface

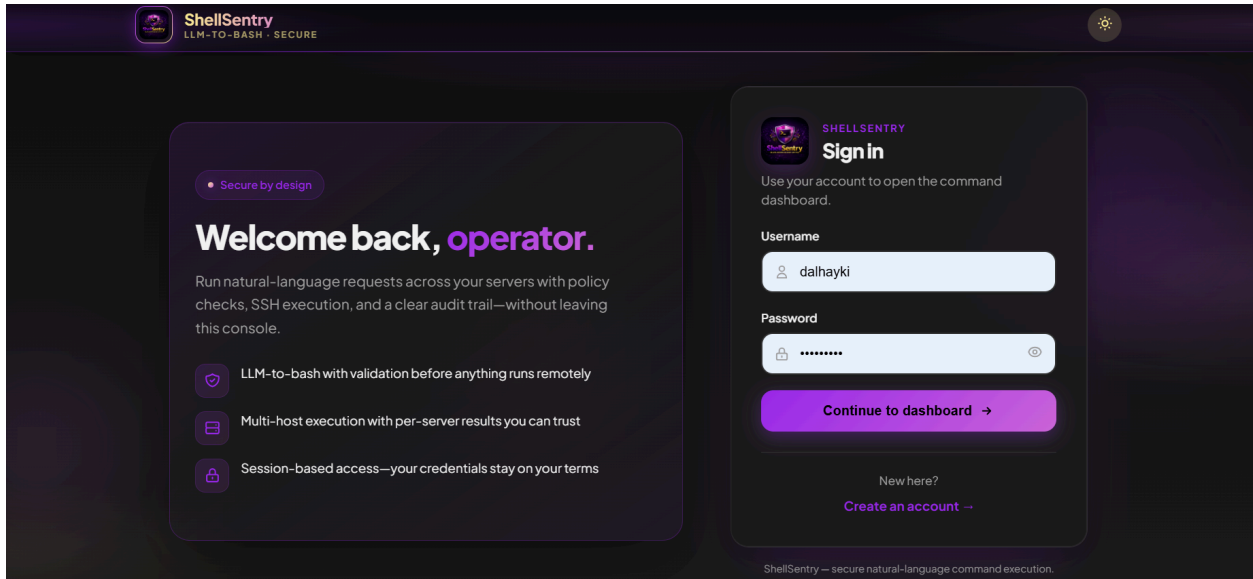


Figure : Log-in Page User Interface

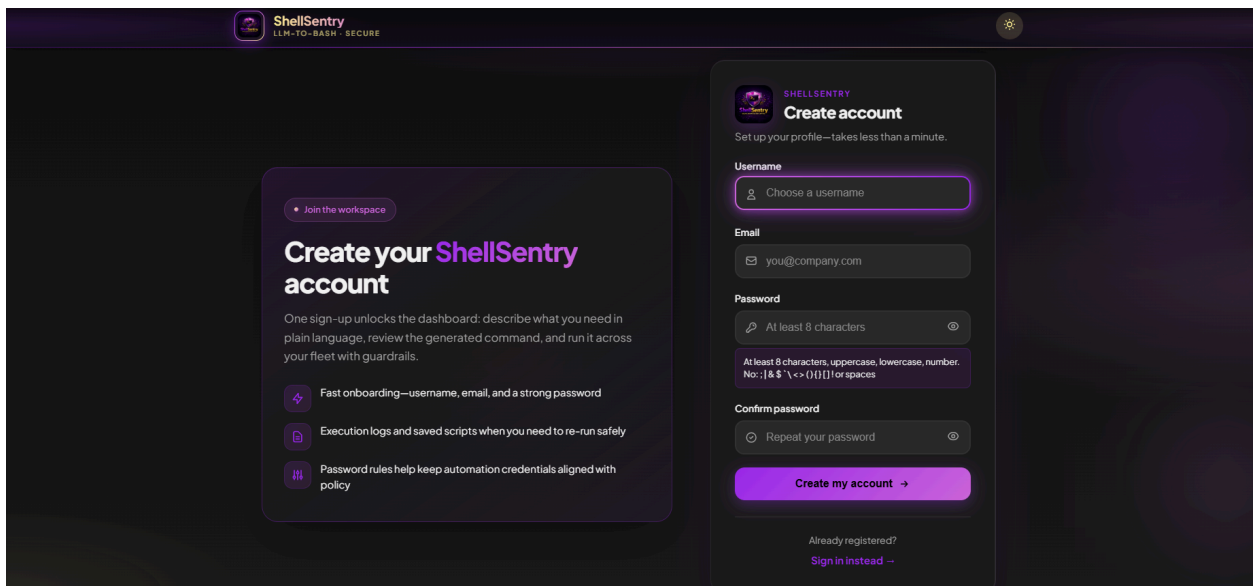


Figure : Sign-up User Interface

5.3.2 Main dashboard: request composition and targets

The dashboard (/dashboard, dashboard.html, @login_required) is the primary workspace. It presents: (1) a natural-language command text area for the user task; (2) an optional target servers field accepting a comma-separated list of hostnames or addresses; (3) an execute control that submits the form via fetch to /api/execute with JSON body { command, servers }. If the server list is left empty, the client sends an empty array and the backend substitutes default hosts from environment configuration (REMOTE_SERVERS), satisfying multi-server

orchestration when defaults are configured. The dashboard may include example prompts (categorised filters in the client script) to support discoverability and training use cases.

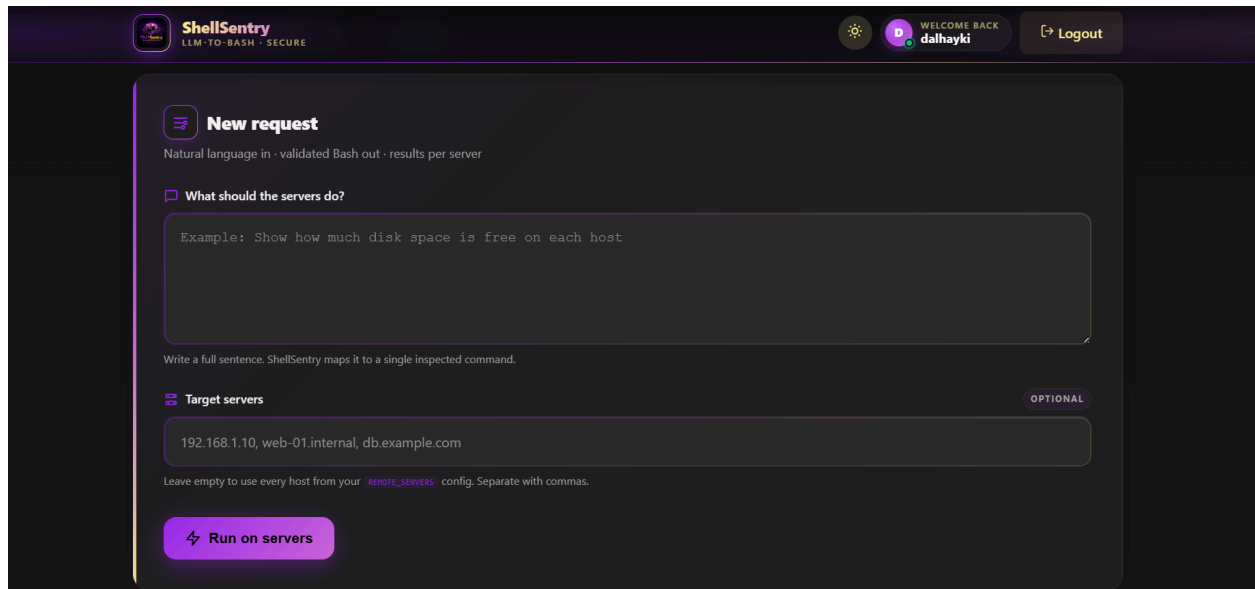


Figure : Dashboard User Interface

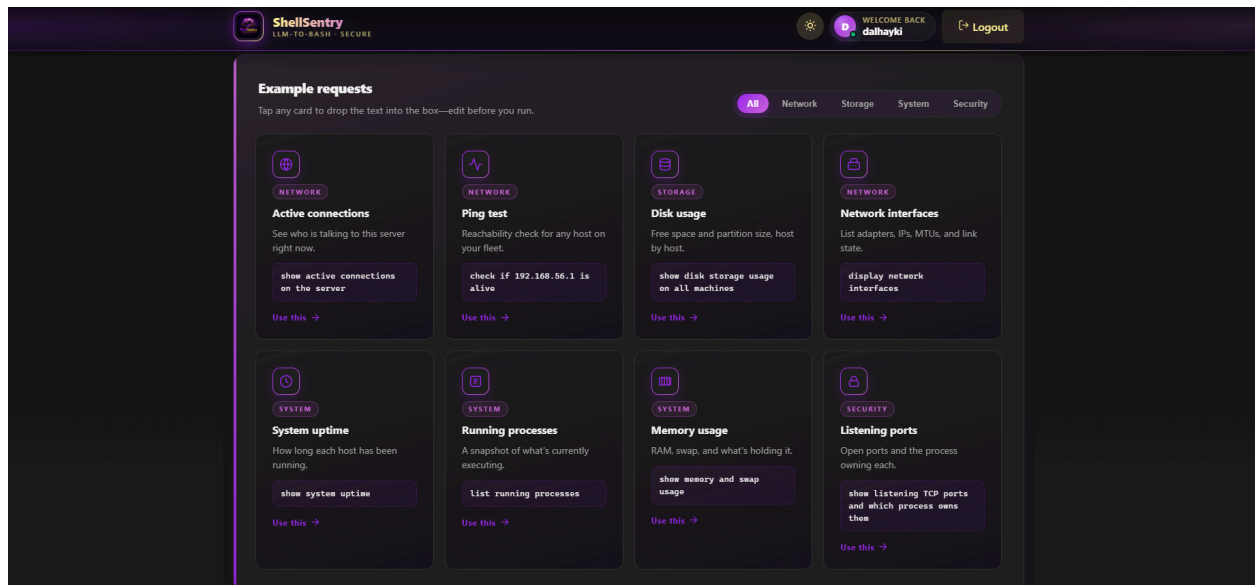


Figure : Option Clickable Execution Command User Interface

5.3.3 Results presentation

On success, the client renders a results stack driven by the JSON payload: a meta strip showing the original request, the generated or logical command label, and per-host success/failure counts derived from the results object. A plain-language summary (`natural_language_summary`)

addresses usability for non-expert readers. When present, an optional AI explanation of the technical report (ai_report_explanation) appears inside a collapsible disclosure; if the explanation is unavailable, a muted fallback message (ai_report_explanation_error) explains that the technical block should be consulted. For script archive flows, script explanation (script_explanation) is shown in the same pattern. The technical report (formatted_report) appears in a collapsible terminal-style panel with a copy-to-clipboard control for lab reporting and appendices. This layout directly supports the audit logging and reporting functional requirement at the presentation layer.

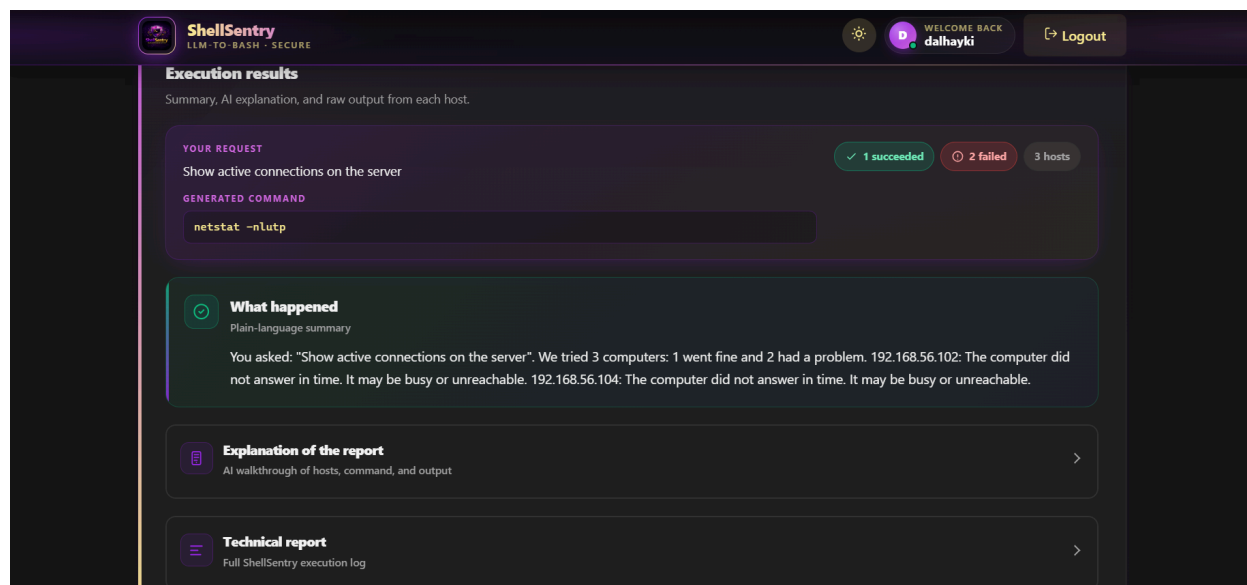


Figure : Result User Interface

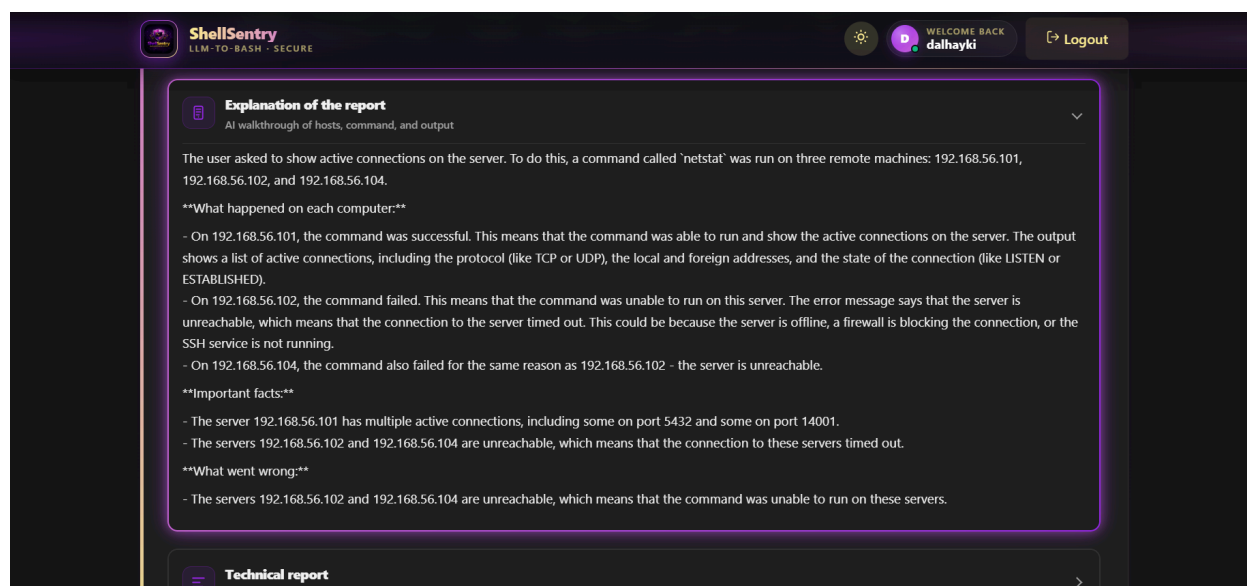


Figure : Explanation User Interface

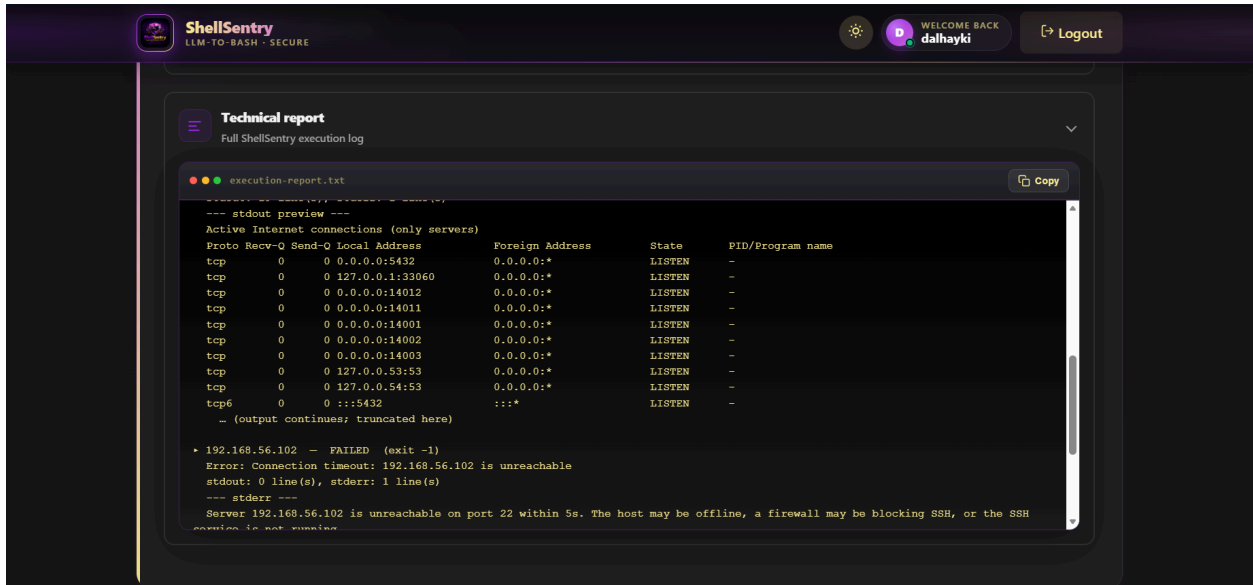


Figure : Technical report User Interface

5.3.4 Error and Blocked-Command Feedback

When the client receives a non-success HTTP response, the interface displays a structured error panel containing both human-readable and technical feedback. The system prioritizes the `natural_language_summary` field to provide clear explanations while optionally displaying validation reasons and backend error details.

If a generated command is rejected by the validation layer, the interface displays the command in a read-only preview labelled as blocked or not executed. The feedback system supports various failure scenarios including invalid commands, read-only policy violations, blocked root execution, SSH authentication failures, unreachable servers, SSH connectivity issues, LLM API failures, invalid cron requests, and internal processing errors.

The interface also provides troubleshooting guidance such as checking server connectivity, SSH credentials, LLM configuration, and application logs. This design ensures consistent and secure communication of validation and execution failures without exposing sensitive information.

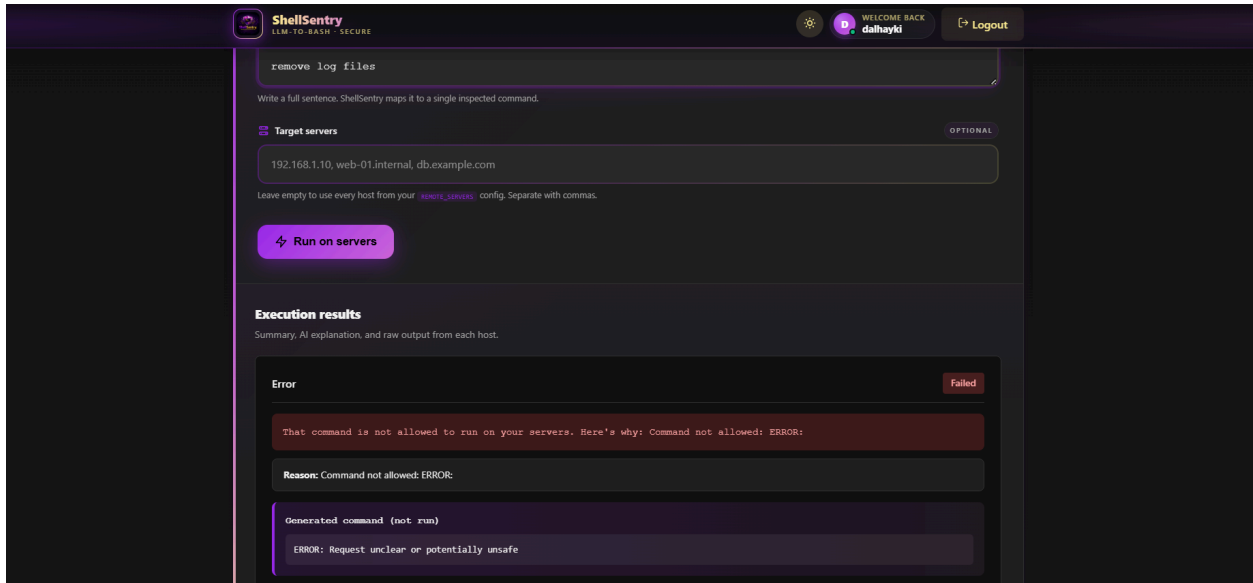
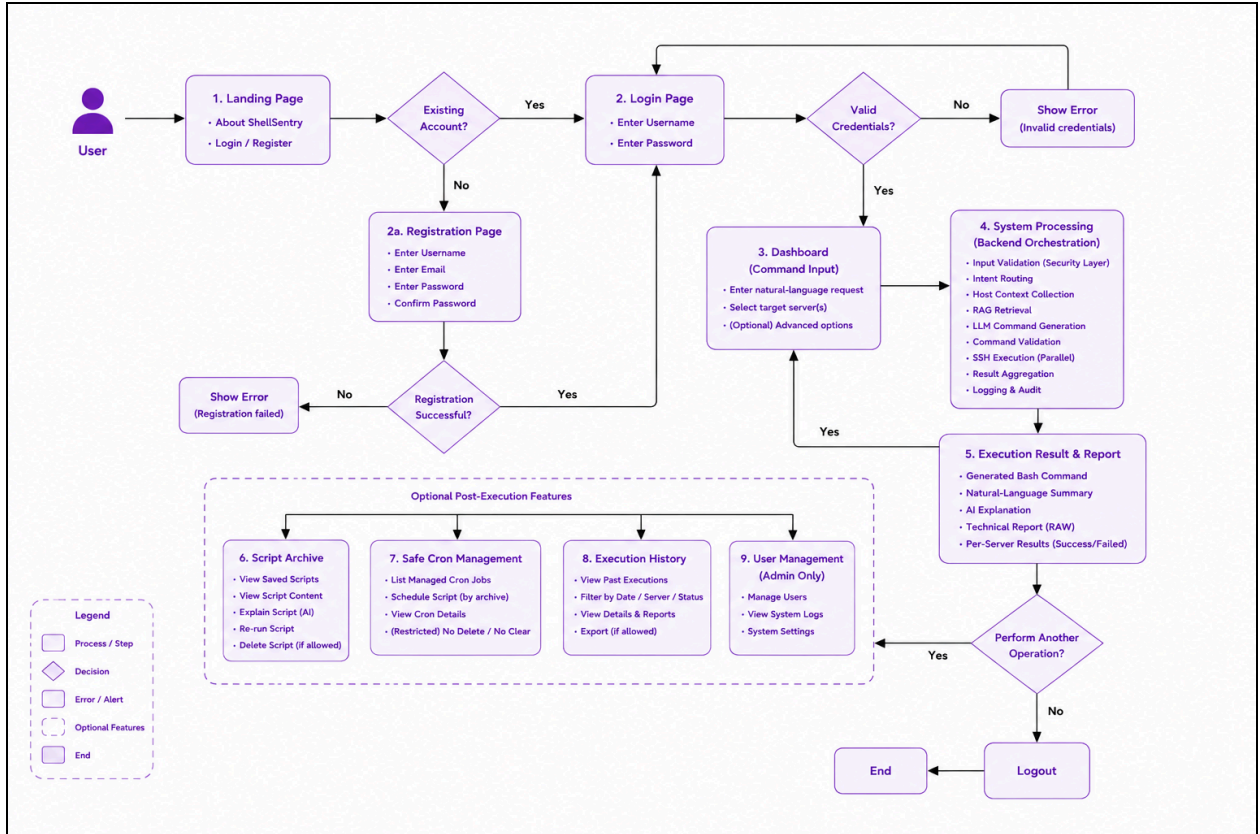


Figure : Error Handling

5.3.5 Navigation and session flow

ShellSentry uses a session-based navigation flow to control access to protected functionality. Unauthenticated users are redirected to the landing, login, or registration pages, while authenticated users are granted access to the dashboard and execution features. After successful login, the user is redirected to the main dashboard where natural-language requests and target servers can be submitted. The navigation structure allows users to move between execution results, archived script operations, and Safe Cron management features when available. User sessions are managed using Flask-Login, which protects sensitive routes and prevents unauthorized access to execution APIs. The logout function securely terminates the active session and redirects the user back to the login page. The overall interaction and navigation process of the system is illustrated in Figure 5, which presents the complete ShellSentry user flow from authentication to command execution and reporting.

User flow



This figure illustrates the end-to-end workflow of the ShellSentry platform, covering user authentication, command execution, backend orchestration, and post-execution management features. The process begins with the landing page, where users either log in or register, followed by credential validation and access to the dashboard for submitting natural-language server commands. The backend processing layer handles security validation, intent routing, RAG retrieval, AI-generated command creation, SSH execution, and result aggregation. After execution, users receive detailed reports and can optionally access features such as script archiving, cron job management, execution history, and administrative user management before logging out or performing additional operations.

5.4 Core Algorithms

This section presents the core algorithms used in ShellSentry. These algorithms coordinate secure natural-language processing, command validation, remote execution, script management, and retrieval-augmented generation. Together, they provide controlled AI-assisted remote administration while enforcing deterministic security validation and operational traceability.

5.4.1 Natural-Language Execution Algorithm

This algorithm represents the primary execution workflow of ShellSentry. It coordinates authentication, input validation, route resolution, host probing, retrieval-augmented generation, command validation, SSH execution, result formatting, and audit logging.

Pseudocode

Algorithm: Natural-Language Execution

REQUIRE authenticated user

READ:

 natural-language request

 target servers

IF request is empty **OR** unsafe:

RETURN validation error

IF no servers selected:

 use default configured servers

Detect execution route using AI and rule checks

IF route is cron operation:

 handle Safe Cron request

RETURN result

IF route is archive operation:

 handle archived scripts

RETURN result

Collect host context from target servers

Retrieve related RAG examples

Generate Bash command using AI

IF generated command is invalid:

RETURN blocked-command response

Normalize validated command

Execute command on remote servers through SSH

Format execution results

Generate AI explanation and summaries

Store execution logs

RETURN:

generated command

execution results

summaries

explanations

IF unexpected error occurs:

RETURN internal server error

This workflow combines deterministic validation with AI-assisted command generation to reduce operational risk while maintaining usability and explainability.

5.4.2 Command Validation Algorithm

This algorithm validates generated Bash commands before remote execution. It applies security policies including blacklist checks, whitelist enforcement, read-only restrictions, and privileged-operation controls.

Pseudocode

Algorithm: Command Validation

READ:

generated command

Remove markdown formatting and shebangs

Split command into segments

Check blacklist patterns

Check whitelist commands

Check restricted operations

IF read-only mode is enabled:

block state-changing commands

IF root execution is disabled:

block sudo and privileged operations

Normalize validated command

RETURN:

valid OR blocked status

This validation layer acts as the primary deterministic security boundary between AI-generated output and remote system execution.

5.4.3 Parallel SSH Execution Algorithm

This algorithm manages concurrent command execution across multiple Linux hosts using SSH connections while isolating host-level failures.

Pseudocode

Algorithm: Parallel SSH Execution

READ:

approved command

target servers

FOR EACH server:

verify SSH connectivity

PARALLEL EXECUTION:

authenticate using configured credentials

execute command through SSH

collect stdout, stderr, and exit code

Store per-server results

Determine overall status:

success

partial success

failed

RETURN execution results

Parallel execution improves scalability and ensures that failure on one host does not prevent execution on other reachable hosts.

5.4.4 Script Archive and Safe Cron Algorithm

This algorithm manages script archival, script reuse, and controlled cron scheduling.

Pseudocode

Algorithm: Script Archive and Safe Cron

READ:

archive or cron request

IF script is multi-line:

save script in archive directory

IF request is list archived scripts:

RETURN archived scripts

IF request is rerun script:

validate archived script

execute validated script

IF request is explain script:

generate AI explanation

IF request is cron scheduling:

validate cron expression

create managed cron entry

IF destructive cron removal detected:

block operation

RETURN operation result

This workflow enables controlled automation reuse while preventing unsafe cron manipulation and unauthorized script execution.

5.4.5 Host Context Collection Algorithm

This algorithm gathers contextual information from remote Linux hosts to support environment-aware command generation.

Pseudocode

Algorithm: Host Context Collection

READ:

target servers

FOR EACH server:

connect through SSH

collect operating-system information

detect running services

inspect listening TCP/UDP sockets

Store collected host context

RETURN host context

The collected context improves command accuracy by allowing the LLM to adapt generated commands to the detected remote environment.

5.4.6 RAG Retrieval Algorithm

This algorithm performs retrieval-augmented generation (RAG) using semantic similarity search over a curated Bash knowledge base.

Pseudocode

Algorithm: RAG Retrieval

READ:

natural-language request

Generate embedding vector

Perform semantic similarity search

Retrieve top-k related command examples

Format examples for prompt grounding

RETURN grounded context

The RAG pipeline reduces hallucinations and improves consistency by grounding the LLM using trusted Bash command examples.

5.5 Mapping to Requirements

This section demonstrates the traceability between the system requirements identified in Chapter 4 and the design components presented in Chapter 5. By mapping both functional and non-functional requirements to specific architectural modules, interfaces, and algorithms, the section shows how the proposed design was intentionally structured to satisfy the project objectives while ensuring security, maintainability, usability, and controlled AI-assisted remote execution.

5.5.1 Functional Requirements Coverage

Chapter 4's functional expectations map directly onto the modular design in Chapter 5: natural-language Bash generation is handled by LLMClient under orchestration in routes/api.py (5.4.1), while host-aware operation uses SSHExecutor.probe_host_context (5.4.5) and RAG grounding uses RagPipeline embedding and similarity search over the curated knowledge base (5.4.6). Deterministic safety is split between SecurityLayer on user text and CommandValidator on generated or archived script content, including normalisation before any remote run (5.4.2). SSH execution, multi-server orchestration, and flexible authentication are realised in SSHExecutor together with Config defaults and credential resolution (5.4.3; dashboard target entry in 5.3.2). Script archive list, rerun, and explain paths, plus Safe Cron list/schedule with blocked destructive removal, combine execution_router, routes/api.py, and SSHExecutor (5.4.4). Audit logging and reporting tie ExecutionLog, executor logging, result_formatter, and optional LLMClient summaries into the JSON the UI displays (5.2, 5.3.5, 5.4.1). Configurable policies (read-only default, root toggle, archive/cron/LLM settings) are expressed in Config and enforced alongside validation (5.2.3, 5.4.1–5.4.2).

5.5.2 Non-Functional Requirements Coverage

Security and defence in depth are distributed across input validation, intent routing, and post-generation policy so that no single layer is solely responsible for safety. Accountability and traceability are supported by ExecutionLog and structured log fields that capture who requested an action, what was generated, which hosts were targeted, and what each host returned. Usability and explainability are addressed through the dashboard layout and the API's natural-language summaries, optional AI explanations, and expandable technical reports (5.3). Reliability under partial failure is supported by parallel SSH with isolated per-host results so one bad host does not block the rest. Maintainability follows from the separation of concerns among

routes, the execution router, validators, the SSH executor, the LLM client, RAG, and formatting modules (5.1), keeping each non-functional concern traceable to a clear part of the design for verification in Chapter 6.

5.6 Chapter summary

Chapter 5 presented the system design of ShellSentry as a bridge between the requirements in Chapter 4 and the implementation and evaluation in Chapter 6. The chapter described a modular, defence-in-depth architecture: Flask routes handle HTTP and authentication; SecurityLayer and execution_router constrain user input and classify intent (including Safe Cron and script-archive paths); RagPipeline and LLMClient supply grounded, host-aware generation on the standard path; CommandValidator enforces deterministic policy before any remote run; and SSHExecutor performs parallel SSH execution with configurable authentication and isolated per-host outcomes. The database design is intentionally narrow—User and ExecutionLog—to support login and auditable execution history while keeping operational state on remote Linux hosts. The interface design covers landing, login, registration, the command dashboard, and rich result and error reporting, including optional AI explanations. Section 5.4 formalised the core algorithms for execution, validation, parallel SSH, script archive and Safe Cron, host probing, and RAG retrieval. Section 5.5 mapped functional and non-functional requirements to these components so that design choices remain traceable. Overall, the design prioritises deterministic security validation, accountable remote execution, controlled AI-assisted automation, and usable reporting within the limits of an educational prototype.

Chapter 6 System implementation and testing

This chapter describes how ShellSentry was implemented as a working prototype and how it was tested against the requirements and design in Chapters 4 and 5. Section 6.1 covers the development environment, technology choices, project layout, configuration, and a feature walk-through supported by screenshots. Section 6.2 presents the evaluation methodology, functional and security testing, performance observations, optional llm model comparisons, a baseline comparison with a representative CLI tool, and a concise strengths and weaknesses analysis linked to Chapter 2.

6.1 System implementation

This section describes the implementation of ShellSentry as a working prototype and explains how the selected technologies, software components, and deployment environment were integrated to satisfy the system requirements defined in Chapter 4 and the design presented in Chapter 5. The implementation emphasizes secure AI-assisted Bash generation, authenticated remote execution, modular architecture, and auditable multi-host orchestration.

6.1.1 Development environment and hardware specifications

ShellSentry was developed and tested on a Windows 11 Home workstation (version 23H2, OS build 22631.6199). The development machine uses an 11th Gen Intel Core i5-1135G7 @ 2.40GHz CPU and 7.8 GB RAM. Storage for development includes the system drive (C: total 405.85 GB, 69.3 GB free) and an additional drive (D: total 50.27 GB, 43.26 GB free). The prototype application was executed using Python 3.13.0 (run via python run.py), and the Flask service was used through the web dashboard and REST API.

For remote execution and evaluation, the test environment used Linux virtual machines hosted in VirtualBox, including Raven Linux instances and a Kali Linux instance. The prototype’s default target configuration (from .env) listed the SSH targets as 192.168.56.101, 192.168.56.102, and 192.168.56.104 (all accessed over SSH on port 22, with pre-flight reachability checks). The LLM integration was configured through an OpenAI-compatible endpoint (LLM_API_BASE_URL=https://api.groq.com/openai/v1) using the model llama-3.1-8b-instant; the API key was provided via environment configuration (not stored in code).

6.1.2 Technology Stack Selection and Justification

Table 6 presents the main technologies and libraries used in ShellSentry together with their implementation roles and justification.

Technology / Tool	Version	Purpose in ShellSentry	Justification
Python	3.x	Main programming language	Provides strong library support for web development, AI integration, SSH communication, and rapid prototyping.
Flask	3.0.0	Web framework and REST API	Lightweight framework suitable for modular web applications and API routing.

Flask-Login	0.6.3	Session management and authentication	Protects execution routes and ensures only authenticated users can access system functionality.
Flask-SQLAlchemy	3.1.1	ORM and database integration	Simplifies database operations and supports persistent audit logging.
SQLite	Default	Prototype database	Lightweight database suitable for educational prototypes and local deployment.
Werkzeug	3.0.1	Password hashing and verification	Improves credential security through hashed password storage.
Requests	2.31.0	HTTP communication with LLM APIs	Supports OpenAI-compatible API communication and flexible provider integration.
Sentence-Transformers	Latest	Embedding generation for RAG	Generates semantic embeddings for similarity-based retrieval.
all-MiniLM-L6-v2	Default model	Embedding model	Lightweight embedding model suitable for fast semantic search.
FAISS (faiss-cpu)	Latest	Similarity search engine	Performs efficient top-k retrieval over the command knowledge base.
Paramiko	3.4.0	SSH communication and execution	Enables secure remote execution, host probing, and multi-server orchestration.

python-dotenv	Latest	Environment configuration loading	Loads SSH, LLM, and security settings from <code>.env</code> files without hardcoding secrets.
HTML / CSS / JavaScript	—	Frontend implementation	Provides the web interface, dashboard, result display, and client-side interaction.
Jinja2 Templates	Built-in with Flask	Server-rendered frontend pages	Simplifies integration between Flask backend and frontend pages.
openai	1.6.1	LLM-related dependency	Included in the project dependencies to support OpenAI ecosystem compatibility (LLM calls are implemented via OpenAI-compatible HTTP patterns).
bcrypt	4.1.2	Cryptography dependency (commonly for password hashing stacks)	Part of the dependency set for secure hashing; password hashing is handled through the Werkzeug hashing utilities in the prototype.

Implementation Alignment:

Together, these technologies form the required ShellSentry workflow: authenticated request handling (Flask and Flask-Login), deterministic safety enforcement (validation and security modules), grounded AI generation (RAG and LLM integration), and auditable multi-host execution (Paramiko and ExecutionLog). These implementation choices directly support the system requirements and design goals defined in Chapters 4 and 5.

6.1.3 Project structure and module organization

ShellSentry is structured to separate the web layer, AI pipeline, security policy, and remote execution logic. `run.py` starts the Flask server, while `src/app.py` loads configuration and registers

routes. User pages (login, registration, dashboard) are implemented in `src/routes/pages.py`, and the main execution orchestration (`POST /api/execute`) is handled by `src/routes/api.py`. The defence-in-depth pipeline is split across `src/security.py` and `src/command_validator.py`, while AI generation and grounding are implemented in `src/llm_client.py` and `src/rag_pipeline.py`. Remote probing and SSH execution are implemented in `src/ssh_executor.py`, with data models for authentication and audit logging defined in `src/models.py`. The UI is built using templates in `templates/` and client-side behaviour in `static/` (JS/CSS), while audit data is stored locally in the SQLite database under `instance/`.

```
ShellSentry/  
├─ run.py  
├─ requirements.txt  
├─ .env (local config/secrets; exclude from report upload)  
├─ instance/  
│   └─ shellsentry.db (SQLite audit DB)  
├─ src/  
│   ├── app.py  
│   ├── routes/  
│   │   ├── api.py  
│   │   └─ pages.py  
│   ├── execution_router.py  
│   ├── services.py  
│   ├── security.py  
│   ├── intents.py  
│   ├── command_validator.py  
│   ├── ssh_executor.py  
│   ├── llm_client.py  
│   ├── rag_pipeline.py  
│   ├── result_formatter.py  
│   └─ models.py  
├─ templates/  
│   ├── index.html  
│   ├── login.html  
│   ├── register.html  
│   ├── dashboard.html  
│   └─ errors/error.html  
└─ static/  
    ├── css/style.css  
    └─ js/dashboard.js
```

Figure 15.

6.1.4 Configuration and deployment behaviour

ShellSentry is configured through environment variables loaded at runtime using `.env` (via `python-dotenv` and `src/config.py`). The prototype uses the local SQLite database by default (`DATABASE_URL=sqlite:///shellsentry.db`), enabling persistent storage of authentication data and the execution audit trail. LLM integration is configured with `LLM_API_TYPE=openai`, `LLM_API_BASE_URL=https://api.groq.com/openai/v1`, and `LLM_MODEL=llama-3.1-8b-instant`, so the system can call an OpenAI-compatible chat/completions endpoint via HTTP. Remote SSH behaviour is driven by `SSH_USER=hero` and the target host list in `REMOTE_SERVERS` (Raven Linux and Kali test hosts), while host-specific SSH overrides are supported through `SERVER_CREDENTIALS` (so different usernames/credentials can be used per host without changing code). The deployment prototype starts the Flask service using `python run.py`, which runs the app in debug mode and binds to 0.0.0.0 on port 5001 for dashboard/API access. Security and feature behaviour are also controlled by configuration flags in `src/config.py`. In the `.env` used for development/testing, `READ_ONLY_EXECUTION=false` and `ALLOW_ROOT_EXECUTION=true` were enabled to allow broader command execution during implementation and demonstrations; however, deterministic safety still remains enforced by the layered input and command validation logic in `SecurityLayer` and `CommandValidator`. For script archive and Safe Cron features, the prototype supports configurable names and limits via `SCRIPT_ARCHIVE_DIR_NAME`, `SCRIPT_ARCHIVE_MAX_LIST`, `SAFE_CRON_MODE`, and `SAFE_CRON_TAG_PREFIX`; where these values are not overridden in `.env`, the design uses the defaults defined in `src/config.py` (e.g., managed Safe Cron filtering and a dedicated remote script archive directory).

6.1.5 Implemented features vs Chapter 5 design

ShellSentry was implemented according to the modular design described in Chapter 5. The web application and authentication flow (landing/login/register and protected access to execution) were implemented using Flask and Flask-Login, while the main execution orchestration was implemented in `src/routes/api.py` for POST `/api/execute`. The defence-in-depth routing logic (selection of Safe Cron, script-archive actions, or the standard NL→Bash flow) was implemented using the execution router (`src/execution_router.py`) with explicit policy blocks for disallowed cron-removal intents, matching the intended “route before execution” design. Host-aware generation was implemented in `src/ssh_executor.py` via `probe_host_context`, which collects per-host environment information over SSH. Retrieval grounding was implemented in `src/rag_pipeline.py` using sentence-transformer embeddings and FAISS similarity search to supply top-k trusted examples to the LLM prompt. Deterministic safety enforcement was implemented through `src/security.py` (input validation) and `src/command_validator.py` (command cleanup, blacklist checks, whitelist enforcement, read-only and privileged-operation rules, and command normalisation). Approved commands were executed remotely using `src/ssh_executor.py` with reachability pre-flight checks and parallel per-host execution, producing independent per-host results and an overall status. The script-archive workflow (list, re-run, and explain saved scripts) and Safe Cron scheduling (managed entry listing/scheduling and destructive removal blocking) were implemented in the SSH executor layer, consistent with

the design that all archive/cron operations remain policy-gated and validated. Finally, auditability was implemented via persistent entries in the local database through the ExecutionLog model, with each execution stored as success, failed, or partial based on per-host outcomes.

6.2 Evaluation and testing

This section presents the evaluation and testing of the implemented ShellSentry prototype in order to verify that the system satisfies its functional, security, and performance requirements. Testing was conducted in the controlled laboratory environment described in Section 6.1 using repeatable manual test scenarios that reflect realistic usage conditions. The evaluation includes functional testing to verify system behaviour, security testing to assess the effectiveness of implemented protection mechanisms, performance measurements of core operations, comparative evaluation of different LLM configurations, and comparison with an existing real-world solution. The strengths, weaknesses, and relation of the obtained results to the literature reviewed in Chapter 2 are also discussed.

6.2.1 Testing methodology

Testing was carried out in the laboratory environment described in Section 6.1, using manual, repeatable scenarios rather than a full automated test suite. Functional checks verified authenticated access and the main `/api/execute` behaviour for single-host and multi-host runs, including default target selection and structured success/error responses. Security checks validated session protection, early input rejection, post-generation command policy enforcement, and Safe Cron destructive-intent blocking, with results recorded as pass/fail against expected controls. Performance was measured from the browser Network panel using Waiting (TTFB) on successful execute requests, including comparisons across host counts and between successful and rejected responses where recorded. Finally, a controlled two-model comparison was performed on the same prompt and host by changing only `LLM_MODEL` on the same Groq-compatible API, reporting five successful runs per model using median and mean TTFB. These choices prioritise transparent, supervisor-reviewable evidence while acknowledging limitations such as small sample sizes and provider/network variability.

6.2.2 Functional testing

Functional tests verify that the implemented behaviour matches Chapter 4 functional requirements and Chapter 5 design: authentication, `/api/execute` orchestration, host probing, RAG retrieval, LLM generation, command validation, parallel SSH execution, script-archive and Safe Cron branches where used, and structured reporting.

Table 7 — Representative functional test scenarios

ID	Scenario	Steps	Expected	Result	Notes
F1	Unauthenticated access	Open /dashboard without logging in	Redirect to login	Pass	Got 302
F2	Login	Valid credentials on /login	Redirect to dashboard	Pass	
F3	NL → Bash (read-only style query)	Enter benign inspection request; optional server list	200, success: true, per-host stdout / exit code	Pass	Example from logs: active connections → generated netstat -nlutp
F4	Default targets	Leave server field empty; REMOTE_SERVERS populated	Executes on configured defaults	Pass	Execute on all live server configured
F5	Multi-host partial outcome	Mix reachable and unreachable hosts	Reachable hosts return results; unreachable marked failed; overall may be partial	Pass	Logs showed port 22 not reachable on some hosts while another succeeded
F6	Validation rejection	Request unsafe command	400, natural_language_summary explains block; no execution on remotes	Pass	Example: "remove log files" led to validation failure So it prevent execution
F7	Archive list	Natural-language list archived scripts	List payload per host	Pass	
F8	Safe Cron list / schedule	List managed entries; schedule with valid .sh and cron	Managed behaviour only	Pass	
F9	Safe Cron destructive block	Phrasing implying crontab wipe	400, no destructive SSH	Pass	

The screenshot shows the ShellSentry login page on the left and a browser's network tab on the right. The login page has fields for Username and Password, and a 'Continue to dashboard' button. The network tab shows a list of requests for the login attempt.

Name	Status	Type	Initiator	Size	Time	Fulfil...
dashboard	302	document /...	Other	416 B	16 ms	
login?next=%2Fdashboard	200	document	5001/dashboard	12.5 kB	9 ms	
style.css	200	stylesheet	login?next=%2Fdashbo	95.7 kB	14 ms	
logo.webp	200	webp	login?next=%2Fdashbo	267 kB	27 ms	
css2?family=Plus+Jakarta+Sans:ital,wght@0,40...	200	stylesheet	login?next=%2Fdashbo	1.2 kB	215 ms	
data:image/svg+xml...	200	svg+xml	style.css	0 B	0 ms	(memo...
LDloaomQNCsA88c7O9yZ4KMCcOg4Ko20yw...	200	font	css2?family=Plus+Jaka	27.4 kB	36 ms	
css2?family=Inter:ital,opsz,wght@0,14..32,100...	200	stylesheet	about:client:1	718 B	158 ms	
css2?family=DM+Mono:wght@500&display=s...	200	stylesheet	about:client:1	416 B	153 ms	
datafont/woff2:bas...	200	font	login?next=%2Fdashbo	349 kB	946 ms	

10 requests 754 kB transferred 767 kB resources Finish: 1.42 s DOMContentLoaded: 250 ms Load: 1.47 s

F1 : Unauthenticated access

The screenshot shows the ShellSentry 'Command execution dashboard' on the left and a browser's network tab on the right. The dashboard includes a 'New request' section with a text input for a command. The network tab shows requests for the authenticated session.

Name	Status	Type	Initiator	Size	Time	Fulfil...
login	302	document /...	Other	464 B	164 ms	
dashboard	200	document	login	22.4 kB	5 ms	
style.css	200	stylesheet	dashboard:10	95.7 kB	25 ms	
logo.webp	200	webp	dashboard:18	267 kB	21 ms	
dashboard.js	200	script	dashboard:316	16.9 kB	28 ms	
css2?family=Inter:ital,opsz,wght@0,14..32,100...	200	stylesheet	about:client:1	718 B	153 ms	
css2?family=DM+Mono:wght@500&display=s...	200	stylesheet	about:client:1	416 B	149 ms	
datafont/woff2:bas...	200	font	dashboard	349 kB	721 ms	

8 requests 753 kB transferred 755 kB resources Finish: 1.05 s DOMContentLoaded: 174 ms Load: 1.09 s

F2 : Login

ShellSentry
 LLM-TO-BASH - SECURE

WELCOME BACK **dalhayki**

Logout

Summary, AI explanation, and raw output from each host.

YOUR REQUEST
 show ip address

1 succeeded 1 host

GENERATED COMMAND

```
ifconfig
```

What happened
 Plain-language summary
 You asked: "show ip address". On 192.168.56.104, the IP addresses are 192.168.56.104, 255.255.255.0, 192.168.56.255, and 255.0.0.0. You can open the full report below to see the full text from the computer.

Explanation of the report
 AI walkthrough of hosts, command, and output

Technical report
 Full ShellSentry execution log

Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

1,000 ms 2,000 ms 3,000 ms 4,000 ms 5,000 ms 6,000 ms 7,000 ms

Name	Status	Type	Initiator	Size	Time	Fulfile...
execute	200	fetch	dashboard.js:76	8.1 kB	6.53 s	

1 requests 8.1 kB transferred 7.9 kB resources

Console Issues Autofill Search +

top Filter Default levels No Issues 1 hidden

F3 : NL → Bash (read-only style query)

ShellSentry
 LLM-TO-BASH - SECURE

WELCOME BACK **dalhayki**

Logout

Plain-language summary

You asked: "Display network interfaces". On 192.168.56.101, the IP addresses are 192.168.56.101, 255.255.255.0, 192.168.56.255, and 255.0.0.0. On 192.168.56.102, the IP addresses are 192.168.56.102, 255.255.255.0, 192.168.56.255, and 255.0.0.0. On 192.168.56.104, the IP addresses are 192.168.56.104, 255.255.255.0, 192.168.56.255, and 255.0.0.0. You can open the full report below for the complete output from each computer.

Explanation of the report
 AI walkthrough of hosts, command, and output

Technical report
 Full ShellSentry execution log

```

# execution-report.txt
--- stdout preview ---
engine: flags=4193<CP,BROADCAST,SHMING,MULTICAST> mem 1500
inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
inet fe80::a00:27ff:fe00:6641 prefixlen 64 scopeid 0x20<link>
ether 08:00:27:00:66:41 txqueuelen 1000 (Ethernet)
RX packets 7413 bytes 267030 (267.0 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 28789 bytes 3400820 (3.4 MB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

engine: flags=4193<CP,BROADCAST,SHMING,MULTICAST> mem 1500
ether 08:00:27:14:8b:3b txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

192.168.56.104 -> SUCCESS (exit 0)
stdout: 17 line(s), stderr: 0 line(s)
--- stdout preview ---
eth0: flags=4163<UP,BROADCAST,SHMING,MULTICAST> mem 1500
inet 192.168.56.104 netmask 255.255.255.0 broadcast 192.168.56.255
inet fe80::a00:27ff:fe03:7c05 prefixlen 64 scopeid 0x20<link>
ether 08:00:27:83:a7:c3 txqueuelen 1000 (Ethernet)
RX packets 412 bytes 42414 (41.2 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

5,000 ms 10,000 ms 15,000 ms 20,000 ms 25,000 ms 30,000 ms 35,000 ms

Name	Headers	Payload	Preview	Response	Initiator	Timing
dashboard	30			"192.168.56.101": {		
style.css	31			"error": null,		
logo.webp	32			"listening_tcp": "State Recv-Q Send-Q Local Addi		
dashboard.js	33			"listening_tcp_stderr": null,		
css2?family=Interital,opsz:wght@0...	34			"listening_udp": "State Recv-Q Send-Q		
css2?family=DM+Mono:wght@500...	35			"listening_udp_stderr": null,		
datafont/woff2/bas...	36			"running_services": "UNIT		
execute	37			"running_services_stderr": null,		
	38			"success": true,		
	39			"uname_line": "Linux raven 6.8.0-57-generic #59-l		
	40			"uname_stderr": null		
	41			},		
	42			"192.168.56.102": {		
	43			"error": null,		
	44			"listening_tcn": "State Recv-Q Send-Q Local Addi		

8 requests 772 kB transferred 775 kB r

Console Issues Autofill Search +

top Filter Default levels No Issues

F4 : Default targets

ShellSentry
LLM-TO-BASH - SECURE

WELCOME BACK
dalhayki

Logout

YOUR REQUEST
show ip address

1 succeeded 2 failed 3 hosts

GENERATED COMMAND

```
ifconfig
```

What happened
Plain-language summary

You asked: "show ip address". On 192.168.56.104, the IP addresses are 192.168.56.104, 255.255.255.0, 192.168.56.255, and 255.0.0.0. You can open the full report below for the complete output from each computer. 192.168.56.101: The computer did not answer in time. It may be busy or unreachable. 192.168.56.102: The computer did not answer in time. It may be busy or unreachable.

Explanation of the report
AI walkthrough of hosts, command, and output

Technical report
Full ShellSentry execution log

Network

☐ Preserve log ☒ Disable cache No throttling

Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Status	Type	Initiator	Size	Time	Fulfil...
execute	200	fetch	dashboard.js:76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js:76	10.6 kB	12.72 s	

2 requests 18.6 kB transferred 18.3 kB resources

Console Issues Autofill Search

top Filter Default levels No Issues 1 hidden

F5 : Multi-host partial outcome

ShellSentry
LLM-TO-BASH - SECURE

WELCOME BACK
dalhayki

Logout

Error

Failed

That command is not allowed to run on your servers. Here's why: Command not allowed: ERROR:

Reason: Command not allowed: ERROR:

Generated command (not run)
ERROR: Request unclear or potentially unsafe

Tips

- Check your internet connection
- Verify the Flask server is running
- Check if LLM_API_KEY is correctly set in your .env file
- Verify your API key is valid and has credits/quota
- Try refreshing the page and submitting again
- Check server logs for detailed error messages

Network

☐ Preserve log ☒ Disable cache No throttling

Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Status	Type	Initiator	Size	Time	Fulfil...
execute	200	fetch	dashboard.js:76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js:76	10.6 kB	12.72 s	
execute	200	fetch	dashboard.js:76	3.4 kB	6.25 s	
execute	400	fetch	dashboard.js:76	471 B	7.13 s	

4 requests 22.5 kB transferred 21.8 kB resources

Console Issues Autofill Search

top Filter Default levels No Issues 1 hidden

F6 : Validation rejection

ShellSentry
LLM-TO-BASH · SECURE

WELCOME BACK **dalhayki**

Logout

did not answer in time. it may be busy or unreachable.

Technical report
Full ShellSentry execution log

execution-report.txt

```

--- stdout ---
stdout: 0 line(s), stderr: 1 line(s)
--- stderr ---
Server 192.168.56.101 is unreachable on port 22 within 5s. The host may be offline, a
firewall may be blocking SSH, or the SSH service is not running.

• 192.168.56.102 - FAILED (exit -1)
Error: Connection timeout: 192.168.56.102 is unreachable
stdout: 0 line(s), stderr: 1 line(s)
--- stderr ---
Server 192.168.56.102 is unreachable on port 22 within 5s. The host may be offline, a
firewall may be blocking SSH, or the SSH service is not running.

• 192.168.56.104 - SUCCESS (exit 0)
stdout: 1 line(s), stderr: 1 line(s)
--- stdout preview ---
2026-05-07 11:54:06.8234145850 | ShellSentry_2026-05-07_11-30-35_875484135.sh

=====
END OF REPORT

```

Filter

Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Status	Type	Initiator	Size	Time	Fulfile...
execute	200	fetch	dashboard.js:76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js:76	10.6 kB	12.72 s	
execute	200	fetch	dashboard.js:76	3.4 kB	6.25 s	

3 requests 22.0 kB transferred 21.5 kB resources

Console Issues Autofill Search +

top Filter Default levels No Issues 1 hidden

F7 : Archive list

ShellSentry
LLM-TO-BASH · SECURE

WELCOME BACK **dalhayki**

Logout

problem: 192.168.56.101: the computer did not answer in time. it may be busy or unreachable.
192.168.56.102: The computer did not answer in time. It may be busy or unreachable.

Technical report
Full ShellSentry execution log

execution-report.txt

```

--- stdout ---
Server 192.168.56.101 is unreachable on port 22 within 5s. The host may be offline, a
firewall may be blocking SSH, or the SSH service is not running.

• 192.168.56.102 - FAILED (exit -1)
Error: Connection timeout: 192.168.56.102 is unreachable
stdout: 0 line(s), stderr: 1 line(s)
--- stderr ---
Server 192.168.56.102 is unreachable on port 22 within 5s. The host may be offline, a
firewall may be blocking SSH, or the SSH service is not running.

• 192.168.56.104 - SUCCESS (exit 0)
stdout: 1 line(s), stderr: 1 line(s)
--- stdout preview ---
* * * * bash "SHOWME/ShellSentryScripts/ShellSentry_2026-05-07_11-30-35_875484135.sh" #
ShellSentryManaged:ShellSentry_2026-05-07_11-30-35_875484135.sh

=====
END OF REPORT

```

Filter

Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

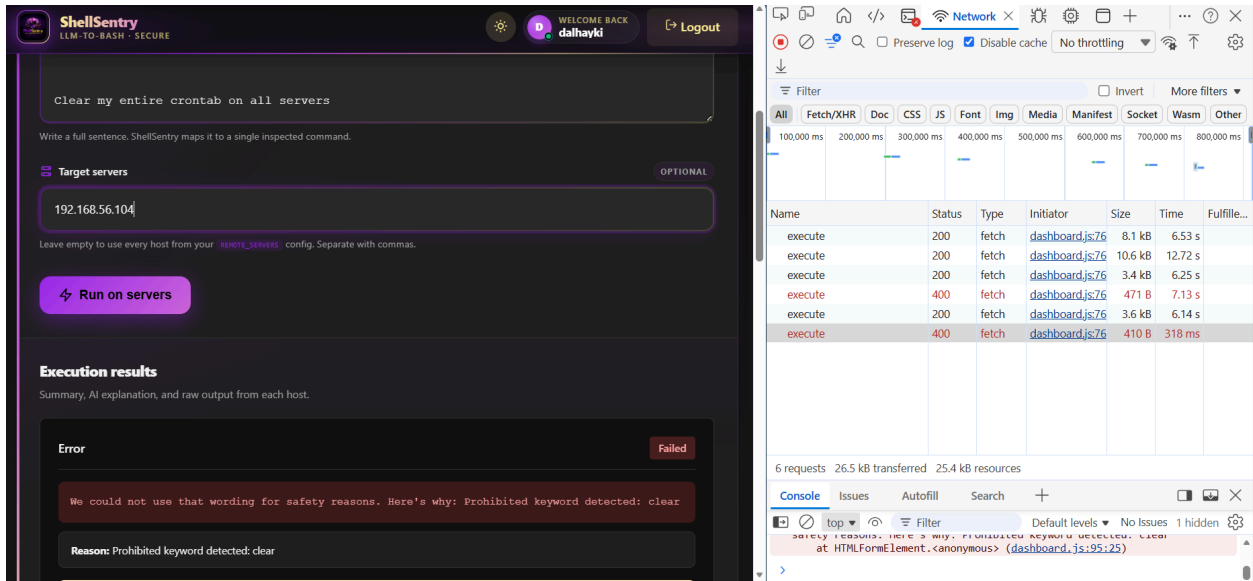
Name	Status	Type	Initiator	Size	Time	Fulfile...
execute	200	fetch	dashboard.js:76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js:76	10.6 kB	12.72 s	
execute	200	fetch	dashboard.js:76	3.4 kB	6.25 s	
execute	400	fetch	dashboard.js:76	471 B	7.13 s	
execute	200	fetch	dashboard.js:76	3.6 kB	6.14 s	

5 requests 26.0 kB transferred 25.1 kB resources

Console Issues Autofill Search +

top Filter Default levels No Issues 1 hidden

F8 : Safe Cron list / schedule



F9 : Safe Cron destructive block

6.2.3 Security testing

Security testing focused on controls implemented in ShellSentry, not third-party penetration testing of unrelated infrastructure. The goal was to show that dangerous paths are blocked and that sensitive configuration is not embedded in source.

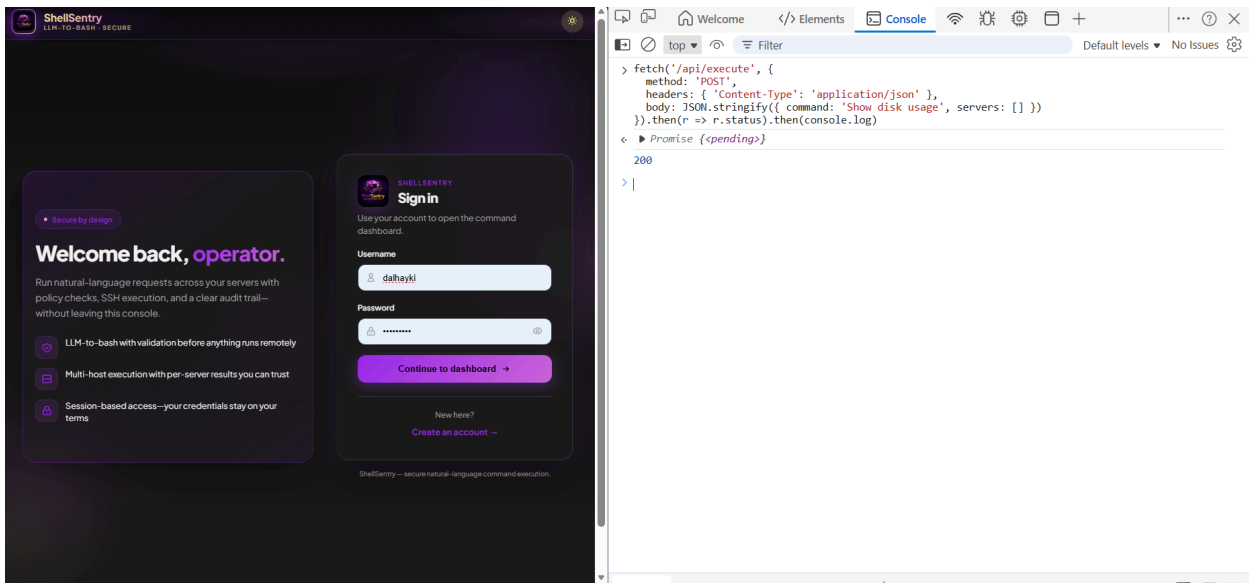
Table 8 — Security-oriented checks

ID	Security objective	Test procedure	Expected outcome	Result	Evidence / notes
S1	Authenticated execution	While logged out, call <code>POST /api/execute</code> (DevTools fetch or direct tool) without session cookies	302 or equivalent unauthenticated rejection	pass	
S2	Session-only dashboard	Open <code>/dashboard</code> without login	Redirect to login	pass	
S3	SecurityLayer input rejection	Submit natural language containing a prohibited	400 with validation message; no SSH	pass	

		keyword / pattern from <code>security.py</code> (use a harmless test string designed to trigger the rule)	execution		
S4	Post-LLM deterministic block	Submit NL that yields a non-whitelisted or blacklisted command (or known-bad operator phrasing)	400; optional display of generated command (not run); no successful remote run	pass	Example pattern: destructive maintenance wording
S5	Read-only enforcement (if <code>READ_ONLY_EXECUTION=true</code>)	Attempt an NL task that implies state change (package install, file delete, service restart)	Blocked by <code>CommandValidator</code> / read-only checks	pass	If tested only with <code>READ_ONLY_EXECUTION=false</code> , mark N/A and explain
S6	Privileged execution gate (if <code>ALLOW_ROOT_EXECUTION=false</code>)	Prompt that tends to produce <code>sudo</code> / root-style commands	Blocked when policy forbids	pass	If <code>true</code> , document as relaxed demo risk
S7	Safe Cron anti-wipe	Submit NL clearly requesting crontab clear/remove/wipe for all entries	400 “cron removal blocked” style response; no destructive cron SSH	pass	
S8	Script archive re-validation (if used)	Re-run a saved <code>.sh</code> whose content would be invalid under current policy	400 before execution	pass	
S9	XSS / injection in UI rendering	Use prompts / outputs containing <code><script>...</code> <code></script></code> or HTML-like	Dashboard shows escaped text (no script execution)	pass	<code>escapeHtml</code> in <code>dashboard.js</code>

		fragments			
S10	Secret hygiene	Confirm <code>.env</code> is listed in <code>.gitignore</code> and not present in Git history for submission	Secrets not committed	pass	Redact keys in screenshots
S11	Transport / deployment caveat (<i>documentation test</i>)	Record that prototype uses HTTP on localhost / LAN without TLS	Document as limitation for production	pass	Not a “pass/fail” of code—honest risk statement

T



Network tab showing requests to localhost:5001. The selected request is `login?next=/api/execute` (Status: 200, Method: GET, Type: html).

Preview of the dashboard page:

dashboard

New here?

[Create an account](#)

ShellSentry — secure natural-language command execution.

© ShellSentry since 2026 - Secure LLM-to-Bash Interface

S1 : Authenticated execution

ShellSentry LLM-TO-BASH - SECURE

Secure by design

Welcome back, operator.

Run natural-language requests across your servers with policy checks, SSH execution, and a clear audit trail—without leaving this console.

- LLM-to-bash with validation before anything runs remotely
- Multi-host execution with per-server results you can trust
- Session-based access—your credentials stay on your terms

Sign in

Use your account to open the command dashboard.

Please log in to access this page.

Username

Your username

Password

Your password

Continue to dashboard →

New here?

[Create an account](#)

ShellSentry — secure natural-language command execution.

Network tab showing requests to localhost:5001. The selected request is `login?next=%2Fdashboard` (Status: 200, Method: GET, Type: html).

S2 : Session-only dashboard

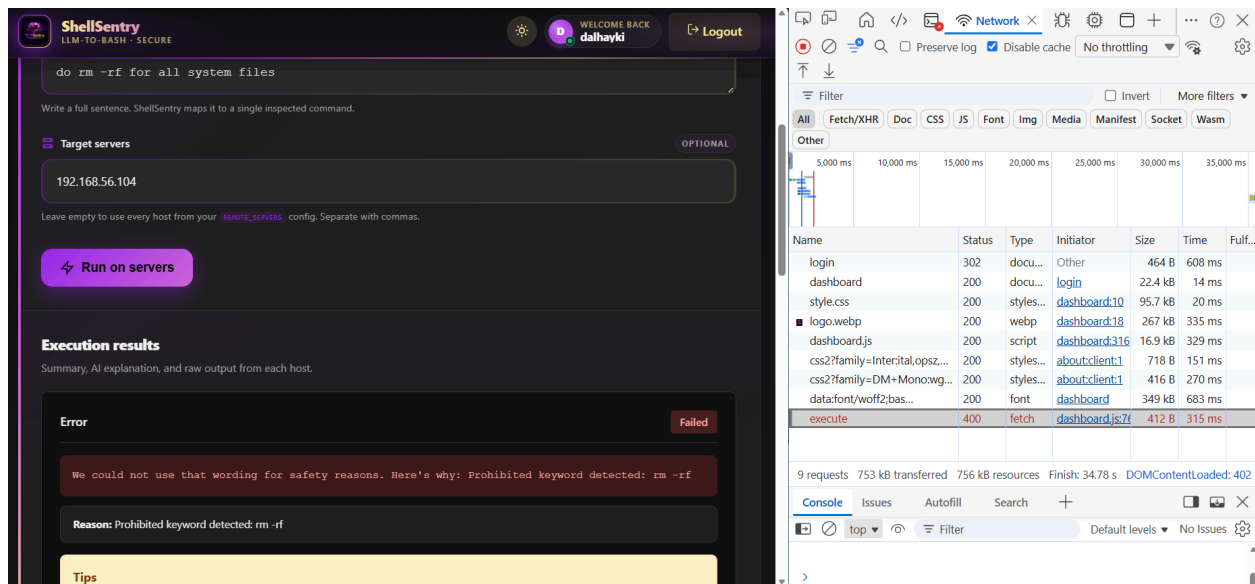


Figure 23 : S3 : SecurityLayer input rejection

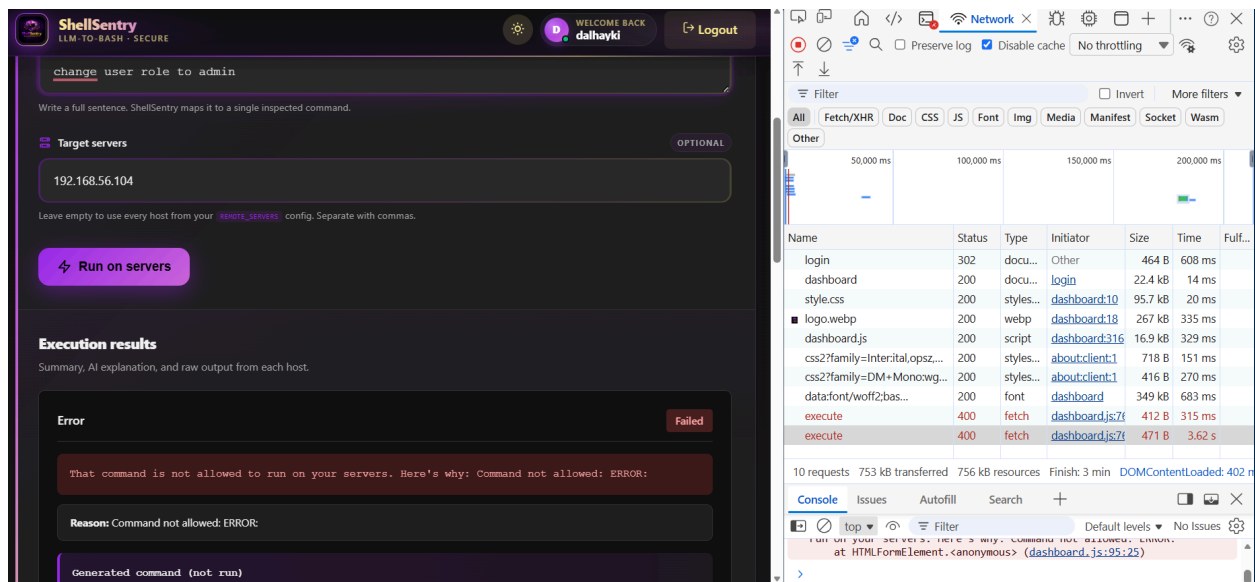


Figure 24 : S4 : Post-LLM deterministic block

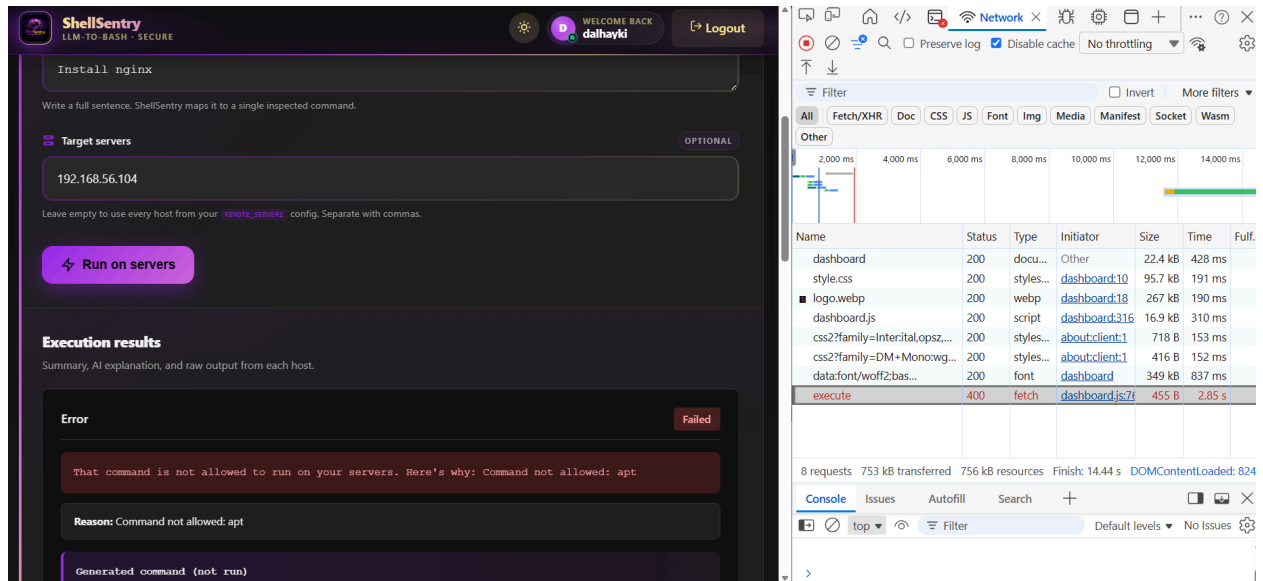
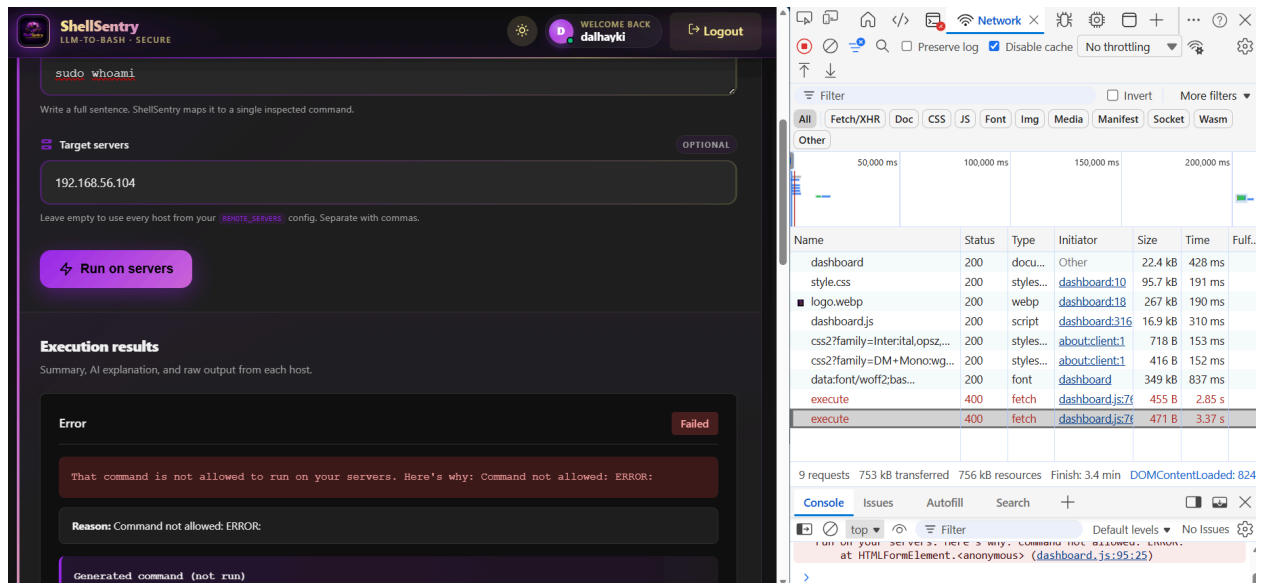


Figure 25 : S5 : Read-only enforcement (if `READ_ONLY_EXECUTION=true`)



S6 : Privileged execution gate (if `ALLOW_ROOT_EXECUTION=false`)

ShellSentry
LLM-TO-BASH - SECURE

WELCOME BACK
dalhayki

Logout

Run on servers

Execution results

Summary, AI explanation, and raw output from each host.

Error

Failed

Removing or clearing crontab entries is blocked by policy. You can try: You can only list or schedule managed ShellSentry script entries.

Tips

- Check your internet connection
- Verify the Flask server is running
- Check if LLM_API_KEY is correctly set in your .env file
- Verify your API key is valid and has credits/quota
- Try refreshing the page and submitting again
- Check server logs for detailed error messages

Network

Preserve log Disable cache No throttling

Filter

Other

5,000 ms 10,000 ms 15,000 ms 20,000 ms

Name	Status	Type	Initiator	Size	Time	Fulf..
login	302	docu...	Other	464 B	495 ms	
dashboard	200	docu...	login	22.4 kB	19 ms	
style.css	200	styles...	dashboard:10	95.7 kB	11 ms	
logo.webp	200	webp	dashboard:18	267 kB	321 ms	
dashboard.js	200	script	dashboard:316	16.9 kB	319 ms	
css2?family=Inter:ital,opsz...	200	styles...	about:client:1	718 B	185 ms	
css2?family=DM+Mono:wght...	200	styles...	about:client:1	416 B	155 ms	
data:font/woff2:bas...	200	font	dashboard	349 kB	714 ms	
execute	400	fetch	dashboard.js:71	419 B	1.41 s	

9 requests 753 kB transferred 756 kB resources Finish: 19.89 s DOMContentLoaded: 415

Console Issues Autofill Search

top Filter

S7 : Safe Cron anti-wipe

ShellSentry
LLM-TO-BASH - SECURE

WELCOME BACK
dalhayki

Logout

re execute ShellSentry_2026-09-07_11-30-39_875484135.sh

Write a full sentence. ShellSentry maps it to a single inspected command.

Target servers

192.168.56.104

OPTIONAL

Leave empty to use every host from your `hosts_services` config. Separate with commas.

Run on servers

Execution results

Summary, AI explanation, and raw output from each host.

Error

Failed

This saved script is not allowed to run under current restrictions. Here's why: Read-only mode: redirecting output to a file is not allowed

Reasons: Read-only mode: redirecting output to a file is not allowed

Tips

- Check your internet connection
- Verify the Flask server is running
- Check if LLM_API_KEY is correctly set in your .env file
- Verify your API key is valid and has credits/quota
- Try refreshing the page and submitting again
- Check server logs for detailed error messages

Network

Preserve log Disable cache No throttling

Filter

Other

10,000 ms 20,000 ms 30,000 ms 40,000 ms 50,000 ms

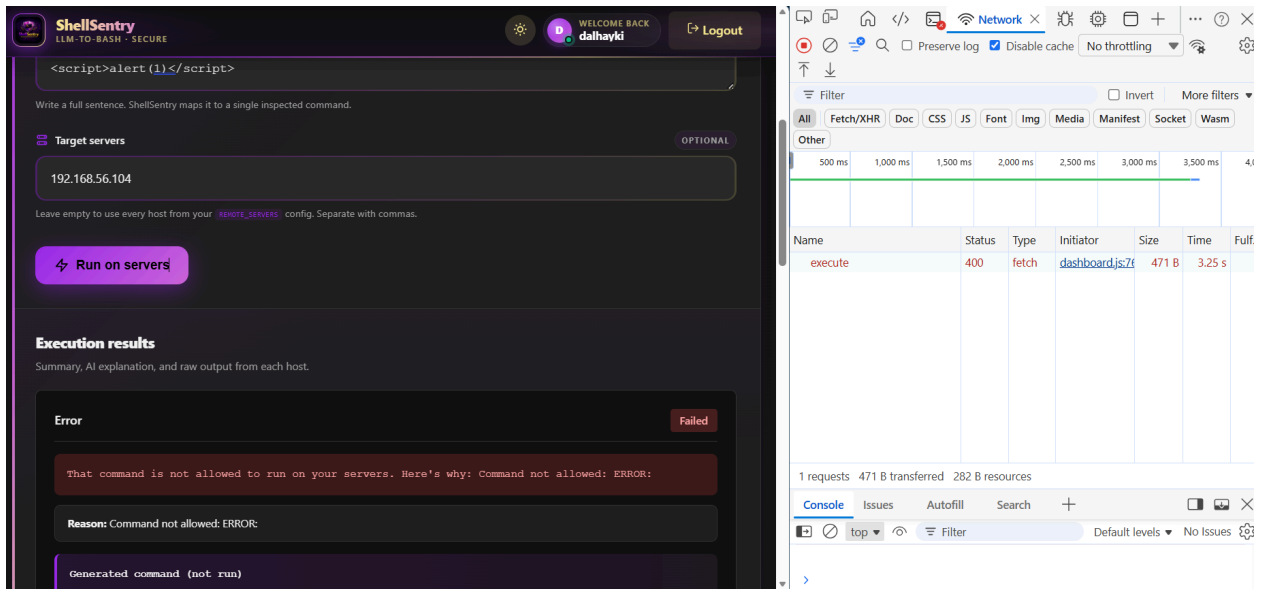
Name	Status	Type	Initiator	Size	Time	Fulfill...
login	302	documen...	Other	464 B	208 ms	
dashboard	200	document	login	22.4 kB	16 ms	
style.css	200	stylesheet	dashboard:10	95.7 kB	24 ms	
logo.webp	200	webp	dashboard:18	267 kB	27 ms	
dashboard.js	200	script	dashboard:316	16.9 kB	25 ms	
css2?family=Inter:ital,opsz,wght@0,14,3...	200	stylesheet	about:client:1	718 B	235 ms	
css2?family=DM+Mono:wght@500&dis...	200	stylesheet	about:client:1	416 B	234 ms	
data:font/woff2:bas...	200	font	dashboard	349 kB	912 ms	
execute	200	fetch	dashboard.js:76	2.0 kB	1.99 s	
execute	400	fetch	dashboard.js:76	497 B	2.06 s	

10 requests 755 kB transferred 757 kB resources Finish: 53.70 s DOMContentLoaded: 265 ms Load: 1.45 s

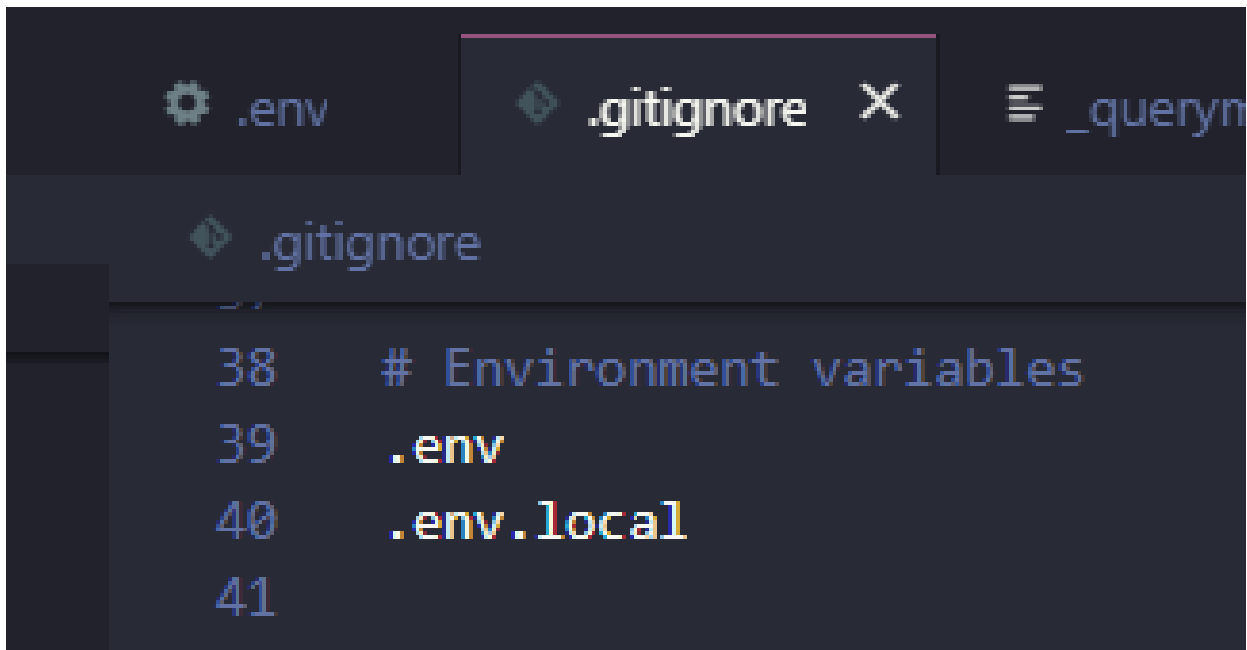
Console Issues Autofill Search

top Filter

S8 : Script archive re-validation



S9 : XSS / injection in UI rendering



S10 : Secret hygiene

6.2.4 Performance measurement

Performance was measured on the development workstation described in Section 6.1.2, using Mozilla Firefox Developer Tools (Network panel). For each run, the POST /execute entry was selected and the Timings tab was used to read Waiting (TTFB)—the interval from sending the request until the first byte of the response arrives. This value is treated as a practical indicator of

server-side processing time for the ShellSentry pipeline (routing, probing, retrieval, LLM calls, validation, SSH execution, and response assembly), even though it also includes a small browser/network overhead component. Very large Queued values (sometimes shown in minutes) were not interpreted as request latency. Those figures mainly reflect how long the request row sat in the Network timeline relative to the overall recording session when many requests accumulate across repeated tests. Therefore, the analysis relies on per-request Waiting (TTFB) rather than Queued duration. During these measurements, the LLM was configured as llama-3.1-8b-instant, accessed through the OpenAI-compatible HTTP endpoint defined in the project environment (e.g., Groq-compatible LLM_API_BASE_URL in .env).

Table 9 summarises the measured scenarios:

Test ID	Scenario	HTTP outcome	Runs (n)	TTFB (Waiting) : min–max	Mean (s)	Median (s)	Notes
P1	Three target hosts	200	5	4.29 – 45.22	≈ 23.10	≈ 12.81	Tail latency on some runs (~45 s).
P2	One target host	200	5	1.66 – 2.78	≈ 2.03	≈ 1.82	Stable “single-host” benchmark set.
P3a	Successful execution (<i>paired with P3b</i>)	200	2	2.98 – 3.65	≈ 3.32	≈ 3.32	For two samples, mean and median coincide with the midpoint $(2.98+3.65)/2$ $(2.98+3.65)/2$.
P3b	Rejected execution	400	2	≈ 1.51 – 1.51	≈ 1.51	≈ 1.51	Small error

							JSON; both failures ~1.51 s TTFB.
--	--	--	--	--	--	--	---

6.2.5 LLM comparison (Groq API models)

We originally considered evaluating locally hosted large language models (for example via Ollama or a similar runtime) to keep inference fully on-machine and reduce dependence on external APIs. In practice, the development workstation used for this project (Intel Core i5-1135G7, 7.8 GB RAM) did not provide enough headroom to run a comparable open-weight model reliably alongside the existing prototype stack (Flask, sentence-transformers embeddings, FAISS, and parallel SSH workloads) without unacceptable slowdowns or instability during interactive testing. For that reason, the comparative evaluation was carried out using Groq’s OpenAI-compatible hosted API, which allows switching models by changing LLM_MODEL while keeping LLM_API_BASE_URL and credentials unchanged.

The comparison used a controlled repeatability setup: the same natural-language prompt and the same single target host (192.168.56.104) were used for each model configuration. End-to-end latency was measured from Firefox → Network → POST /execute → Timings → Waiting (TTFB), recording five successful HTTP 200 executions per model. This metric reflects aggregate server-side work (including LLM inference and remote execution), not an isolated LLM-only benchmark.

Table 6.4 — Two-model comparison under identical prompt/host conditions

LLM_MODEL	n	Median TTFB (s)	Mean TTFB (s)	Notes
llama-3.1-8b-instant	5	4.18	4.67	Same prompt/host; five successful runs

llama-3.3-70b-versatile	5	4.97	7.97	Same prompt/host; five successful runs
-------------------------	---	------	------	--

```
# LLM API Configuration
LLM_API_TYPE=openai
LLM_API_KEY=xxxxxxx
LLM_API_BASE_URL=https://api.groq.com/openai/v1
LLM_MODEL=llama-3.3-70b-versatile
```

Figure : LLM Versatile Configuration

```
# LLM API Configuration
LLM_API_TYPE=openai
LLM_API_KEY=xxxxxxx
LLM_API_BASE_URL=https://api.groq.com/openai/v1
LLM_MODEL=llama-3.1-8b-instant
```

Figure : LLM Instant Configuration

6.2.5.1 Comparison Of Findings

Under this fixed prompt, llama-3.1-8b-instant produced a lower median TTFB (4.18 s) than llama-3.3-70b-versatile (4.97 s), indicating that the smaller model configuration was slightly faster for the typical successful runs in this sample. However, the larger model's mean (7.97 s) exceeds the smaller model's mean (4.67 s) because one run in the five-sample set exhibited substantially higher tail latency, which increases the average more than the median. Both models produced acceptable command variants for the task (primarily `ss -tlnp`, with one `netstat -tlnp` observation for the 8B configuration), which is consistent with LLM sampling variability rather than a functional failure in this controlled check.

6.2.5.2 Comparison Limitations

The comparison is small-sample (five runs per model) and measures browser-observed end-to-end latency rather than token-level inference time split from SSH and formatting. Additionally, hosted inference introduces network and provider-side variability that local inference would reduce but was not feasible on the available hardware for this prototype scope.

6.2.6 Comparison with a Real-World Solution

ShellSentry can be compared with Shell-GPT, which is an AI-powered terminal assistant that converts natural-language requests into shell commands. Both systems aim to simplify command-line interaction using Large Language Models (LLMs). However, the focus of ShellSentry differs by emphasizing secure remote orchestration, deterministic validation, and controlled execution across multiple Linux servers.

Shell-GPT mainly operates as a local command-line assistant for individual users. In contrast, ShellSentry provides a web-based interface with authenticated access, multi-server SSH execution, host-aware generation, script archival, Safe Cron scheduling, and execution audit logging. Additionally, ShellSentry integrates retrieval-augmented generation (RAG), deterministic command validation, read-only execution policies, and root-execution restrictions to reduce the risks associated with AI-generated shell commands.

Although ShellSentry introduces stronger security and orchestration features, Shell-GPT provides direct terminal integration and lightweight CLI interaction that are outside the scope of the current prototype. ShellSentry focuses primarily on controlled remote administration and security-oriented workflows rather than personal terminal productivity.

Table 10 presents a comparison between ShellSentry and Shell-GPT.

Feature	ShellSentry	Shell-GPT
Natural-language-to-Bash generation	Supported	Supported
Web-based dashboard	Supported	Not supported
Direct CLI integration	Not supported	Supported
Multi-server SSH execution	Supported	Limited
Host-aware context collection	Supported	Limited

Retrieval-augmented generation (RAG)	Supported	Not supported
Deterministic command validation	Supported	Limited
Read-only execution mode	Supported	Not supported
Root-execution restriction	Supported	Not supported
Script archival and Safe Cron	Supported	Not supported
Audit logging and reporting	Supported	Limited
AI-generated execution explanations	Supported	Limited

Although ShellSentry is implemented as an educational prototype, the comparison demonstrates how AI-assisted shell generation can be extended with security-focused orchestration, validation, auditability, and controlled automation features beyond traditional AI terminal assistants.

6.2.7 Strengths and Weaknesses

6.2.7.1 Strengths

ShellSentry demonstrates several strengths in both architecture and security design. The system uses a modular pipeline that separates routing, validation, AI generation, SSH execution, and reporting into independent components, improving maintainability and scalability. The platform applies defence-in-depth security through layered input validation, intent routing, deterministic command validation, and configurable execution restrictions. Additionally, host-aware prompting and retrieval-augmented generation (RAG) improve the relevance and

consistency of generated Bash commands by grounding AI output using trusted examples and live host context.

The system also supports parallel SSH execution with reachability pre-checks, allowing efficient multi-server orchestration while isolating host-level failures. Furthermore, the **ExecutionLog** model provides auditability by recording requests, generated commands, target servers, and execution results. Human-readable summaries and AI-generated explanations improve usability for both technical and less-specialised users.

6.2.7.2 Weaknesses

Despite these strengths, the prototype has several limitations. Since command generation relies on a Large Language Model, the generated output remains probabilistic and may still require strict validation and human oversight. The whitelist and restricted-command policies also introduce a trade-off between security and operational flexibility.

Additionally, the current prototype does not implement advanced enterprise features such as full role-based access control (RBAC), immutable SIEM-integrated audit logs, human-in-the-loop approval workflows, or command dry-run simulation. The evaluation environment is limited to a controlled lab setup and primarily English-language interactions. Finally, system performance may vary depending on external LLM API response times, provider rate limits, and virtual-machine resource availability.

6.2.8 Discussion and Comparison with Literature

The evaluation results support the findings discussed in Chapter 2 regarding the limitations and risks of Natural Language to Bash (NL2SH) systems. Previous studies such as ScriptSmith and LLM-as-a-Judge emphasized that Bash generation alone is insufficient without validation and refinement mechanisms. ShellSentry reflects these principles through its layered execution pipeline, deterministic command validation, audit logging, and controlled SSH execution.

The testing phase also confirmed the security concerns highlighted in the literature review, particularly the risk of hallucinated or destructive AI-generated commands. Similar to the observations reported in the NL2SH translation study, unsafe commands were successfully blocked through whitelist, blacklist, read-only, and root-execution restrictions before reaching remote hosts.

In addition, the literature reviewed in Chapter 2 provided limited coverage of secure multi-host orchestration, authenticated remote execution, audit logging, and controlled scheduling. ShellSentry extends beyond pure command generation by integrating host-aware prompting, retrieval-augmented grounding, parallel SSH execution, script archival, and Safe Cron scheduling into a unified security-oriented platform.

6.3 Chapter Summary

This chapter presented the implementation and evaluation of the ShellSentry prototype, demonstrating how the system design from Chapter 5 was translated into a working secure AI-assisted remote administration platform. The implementation integrated authenticated web access, LLM-based Bash generation, retrieval-augmented grounding, deterministic command validation, SSH-based multi-server execution, script archival, and Safe Cron management. The evaluation showed that the prototype successfully met its core functional, security, and performance objectives through functional testing, security validation, performance measurements, and comparative analysis. The results confirmed the effectiveness of the layered security controls, parallel remote execution model, and audit logging approach, while also highlighting prototype limitations such as LLM probabilistic behaviour, lab-scale evaluation constraints, and dependence on external API performance.